

CÔNG NGHỆ PHẦN MỀM



Chương 01: Phần mềm và kỹ nghệ phần mềm

Giảng viên: Ths.Lê Thị Hoàng Anh

Email: anhlth@huce.edu.vn



Nội dung

→ 1. Phần mềm

- Khái niệm cơ bản
- Lịch sử công nghệ phần mềm
- Các đặc trưng của phần mềm
- Thuộc tính của phần mềm
- Tiêu chí của một phần mềm tốt
- Phân loại phần mềm
- Phát triển phần mềm và các thách thức



Nội dung

2. Kỹ nghệ phần mềm

- Định nghĩa
- Đặc điểm về các pha trong vòng đời phần mềm
- Một số mô hình phát triển vòng đời phần mềm
 - Mô hình tuyến tính/vòng đời cổ điển/thác nước
 - Mô hình bản mẫu/chế thử
 - Mô hình gia tăng
 - Mô hình phát triển ứng dụng nhanh RAD
 - Mô hình xoắn ốc, xoắn ốc Win-Win
 - Mô hình linh hoạt Agile và quy trình Scrum



Nội dung

3. Các giai đoạn trong tiến trình kỹ nghệ phần mềm
 - Giai đoạn xác định
 - Giai đoạn phát triển
 - Giai đoạn bảo trì



Khái niệm cơ bản

- Phần mềm máy tính
- Tính môđun (modularity concept)
- Chi tiết hóa dần từng bước (stepwise refinement concept)
- Khái niệm trừu tượng hóa (abstraction concept): về thủ tục, điều khiển, dữ liệu
- Khái niệm che giấu thông tin (information hiding concept)



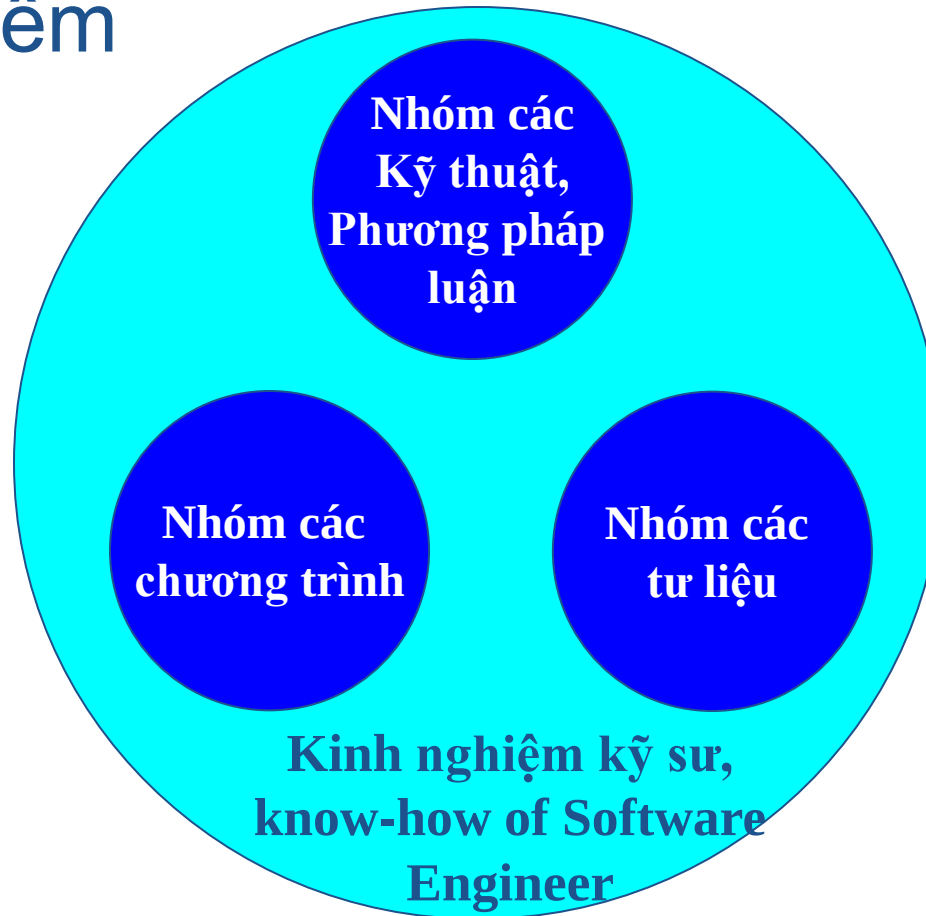
Khái niệm cơ bản

- Phần mềm là sản phẩm do các nhà phát triển phần mềm thiết kế và xây dựng.
- Nó chỉ đến những thành phần không hữu hình của máy tính, nó là thuật ngữ dùng để đối sánh với phần cứng máy tính.
- Phân biệt Software và Programs?



Khái niệm cơ bản

- Phần mềm





Khái niệm cơ bản

- Phần mềm bao gồm:
 - Các lệnh (chương trình) khi được thực hiện trên máy vi tính thì đưa ra hoạt động và kết quả mong muốn.
 - Các cấu trúc dữ liệu hoặc cơ sở dữ liệu làm cho chương trình thao tác thông tin thích hợp.
 - Các tài liệu mô tả thao tác và cách dùng chương trình...



Khái niệm cơ bản

- Phần mềm là **chương trình máy tính** với các **tài liệu liên quan** như yêu cầu phần mềm, mô hình thiết kế, hướng dẫn sử dụng...
- Các sản phẩm phần mềm có thể được thiết kế cho **1 khách hàng cụ thể** nào đó hay cho đối tượng **khách hàng chung chung** (ví dụ: Word, Excel)
- Một phần mềm mới có thể được làm từ đầu hoàn toàn hoặc có thể được điều chỉnh/thay đổi từ 1 phần mềm tổng quát hoặc một phần mềm có sẵn.



Khái niệm cơ bản

- Tính môđun (Modularity) Là khả năng **phân chia phần mềm** thành các môđun ứng với các chức năng, đồng thời cho phép quản lý tổng thể.
 - Phân chia môđun theo chiều sâu
 - Phân chia môđun theo chiều rộng
- Quan hệ giữa các mô đun qua các đối số

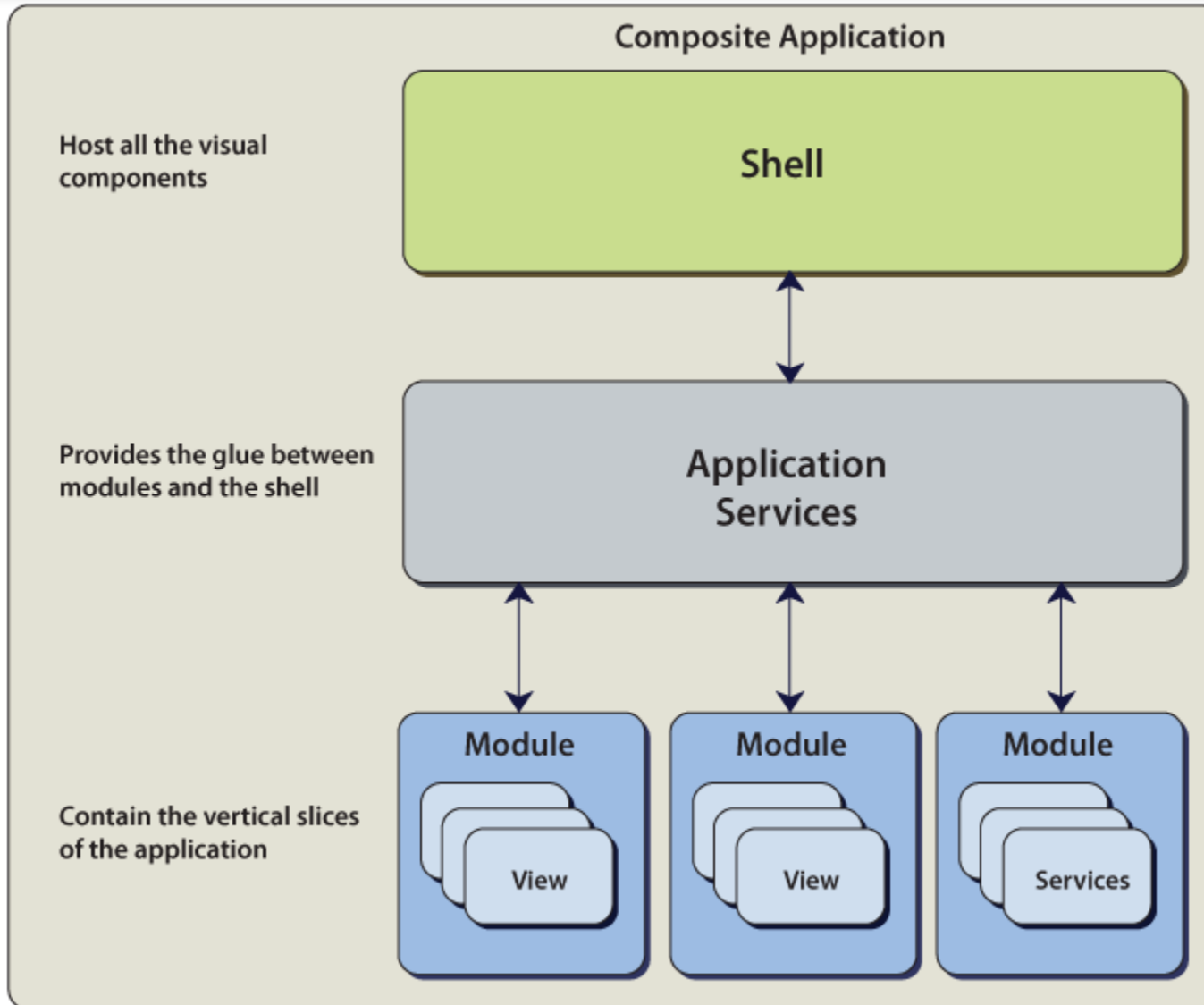


Chuẩn phân chia môđun





Thành phần ứng dụng với nhiều module



Module composition

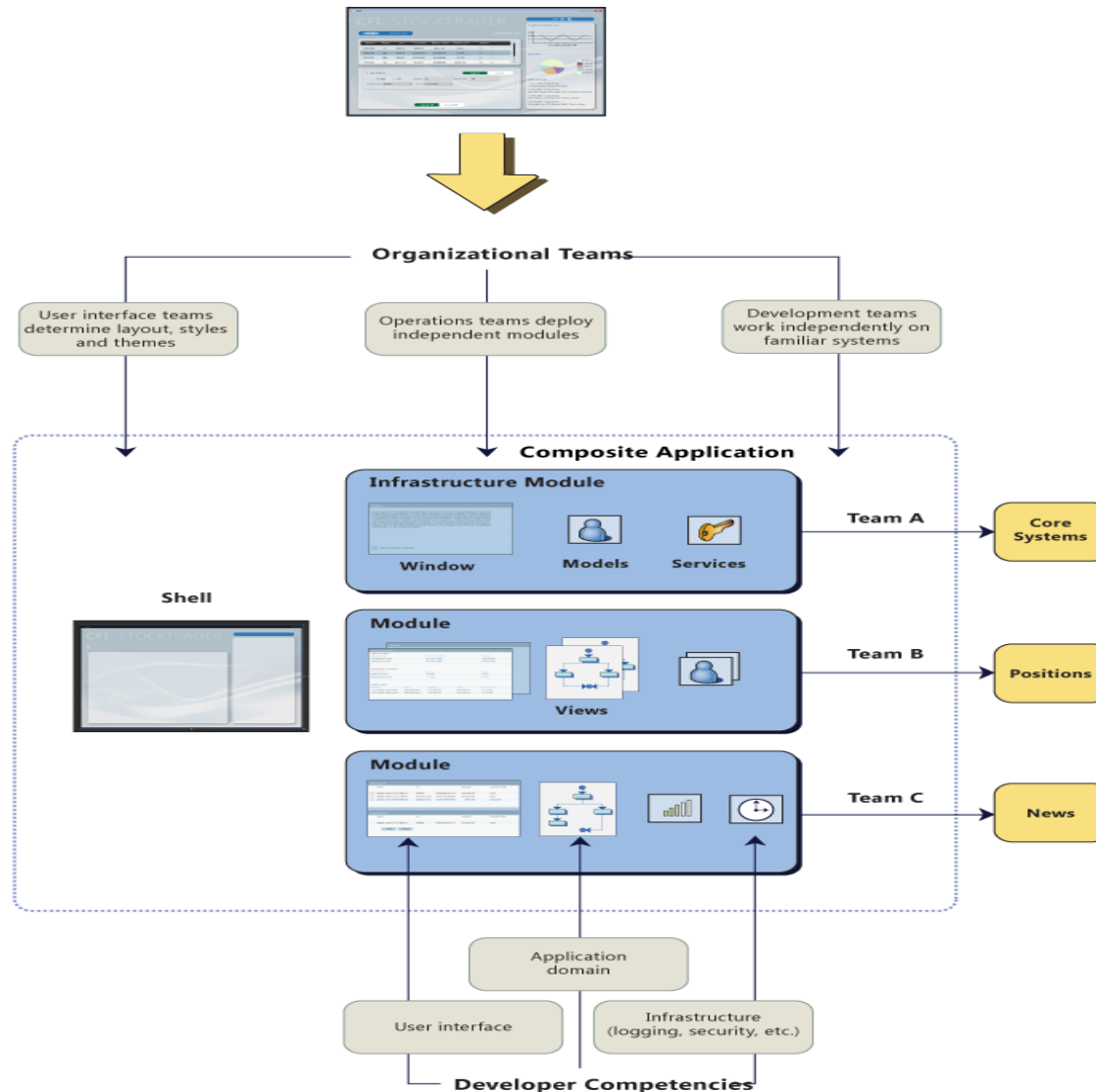


Tại sao chọn thiết kế môđun

- Cấu trúc module đơn giản
- Phát triển module độc lập
- Có thể tải về và sử dụng lại các module từ nhiều nguồn khác nhau.
- Giảm thiểu tối đa thời gian start-up ứng dụng.
- Việc tải về và sử dụng các module dựa theo luật.

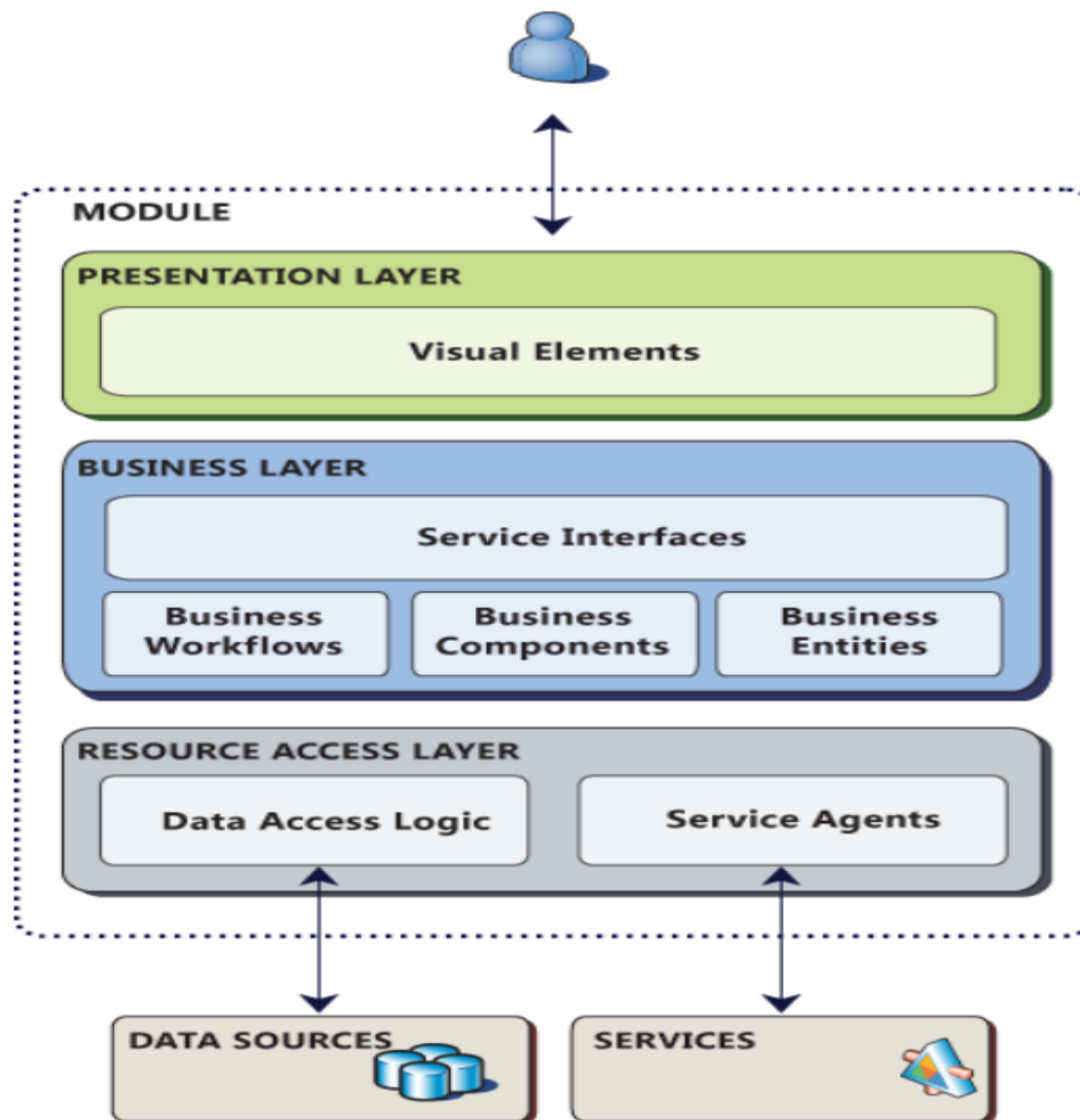


Phát triển module và tổng hợp các nhóm thành phần ứng dụng



Modules and associated teams
of a composite application

Các tầng thành phần trong thiết kế module





Khái niệm cơ bản

- Trừu tượng hóa: Quá trình thiết kế trải qua nhiều mức trừu tượng hóa khác nhau, mức cao, mức vừa và mức thấp.
- Bao gồm trừu tượng hóa thủ tục, trừu tượng hóa dữ liệu và trừu tượng hóa điều khiển.
- Che dấu thông tin: Cần phân rã phần mềm sao cho mỗi môđun ẩn kín đối với các môđun khác.



Lịch sử công nghệ phần mềm

- Thập niên 1940: Các chương trình cho máy tính được viết bằng tay.
- Thập niên 1950: Các công cụ như trình dịch được tối ưu hóa lần đầu tiên ra đời.
- Thập niên 1960- giữa thập niên 1970:
 - Các hệ thống đa chương trình (multi-programming) và đa nhiệm (multi-tasking)
 - Đã đưa ra những khái niệm mới về tương tác người – máy.
 - Những tiến bộ trong lưu trữ trực tuyến dẫn tới thế hệ đầu tiên của các hệ quản trị cơ sở dữ liệu.



Lịch sử công nghệ phần mềm

- 10/1968 tại hội nghị của NATO các chuyên gia phần mềm đã đưa ra thuật ngữ “Khủng hoảng phần mềm”
- Giữa thập niên 1970-1990:
 - Sự phát triển của mạng máy tính và truyền thông, yêu cầu xây dựng các hệ thống phân tán với nhu cầu thâm nhập dữ liệu và đồng bộ “lập tức” làm tăng dần độ phức tạp của hệ thống, đặt ra thách thức lớn với người lập trình.



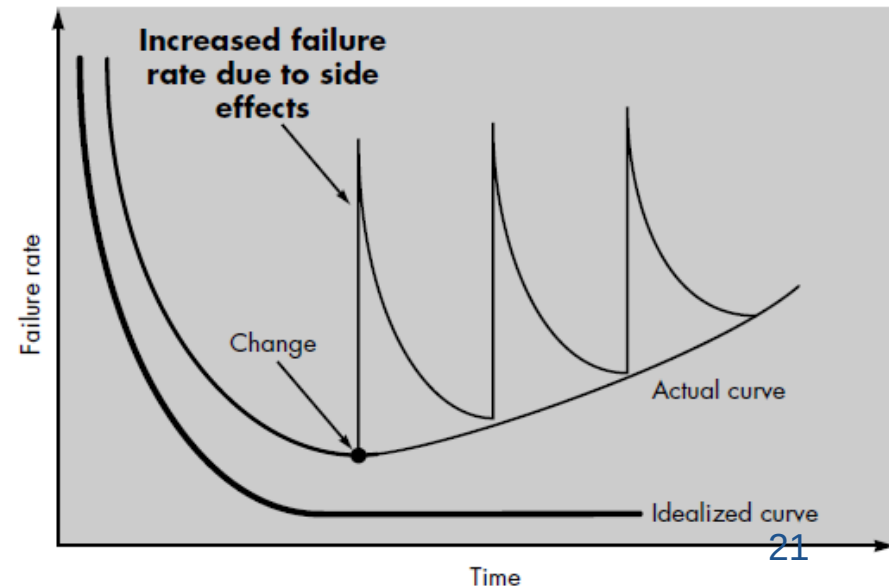
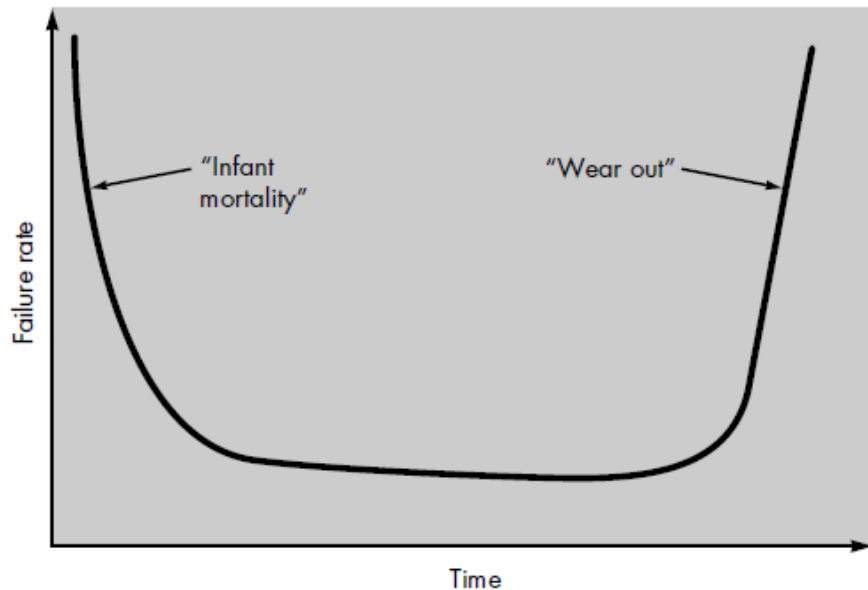
Lịch sử công nghệ phần mềm

- Sự tiến bộ và sự phổ cập sử dụng các máy tính cá nhân, máy trạm khá mạnh.
- Phần cứng giá rẻ trở thành hàng hóa tiêu dùng, chi phí phần mềm tăng (so với chi phí phần cứng)
- 1990- nay: Hệ chuyên gia và phần mềm trí tuệ nhân tạo được ứng dụng vào thực tế. Kỹ nghệ hướng đối tượng nhanh chóng thay thế nhiều cách tiếp cận phát triển phần mềm truyền thống trong các lĩnh vực ứng dụng.



Các đặc trưng của phần mềm

- Phần mềm được phát triển hay kỹ nghệ hóa chứ không phải sản xuất, chế tạo.
- Phần mềm không bị hao mòn như phần cứng mà có xu hướng tốt lên sau mỗi lần phát hiện lỗi và sửa lỗi





Các đặc trưng của phần mềm

- Từ đặc trưng thứ hai chúng ta có thể suy rộng ra:
 - Chức năng phần mềm thường biến hóa, thay đổi theo thời gian và nơi sử dụng.
 - Hiệu ứng lan sóng trong thay đổi phần mềm.
 - Cần khả năng tư duy nhị phân trong xây dựng phát triển phần mềm.



Các đặc trưng của phần mềm

- Phần mềm phần lớn đều được xây dựng theo đơn đặt hàng chứ ít khi được lắp ráp từ những thành phần có sẵn.
- Phần mềm vốn chứa lỗi tiềm tàng, theo quy mô càng lớn thì khả năng chứa lỗi càng cao.
- Lỗi phần mềm dễ được phát hiện bởi người ngoài.
- Phần mềm có thể sao chép rất đơn giản.



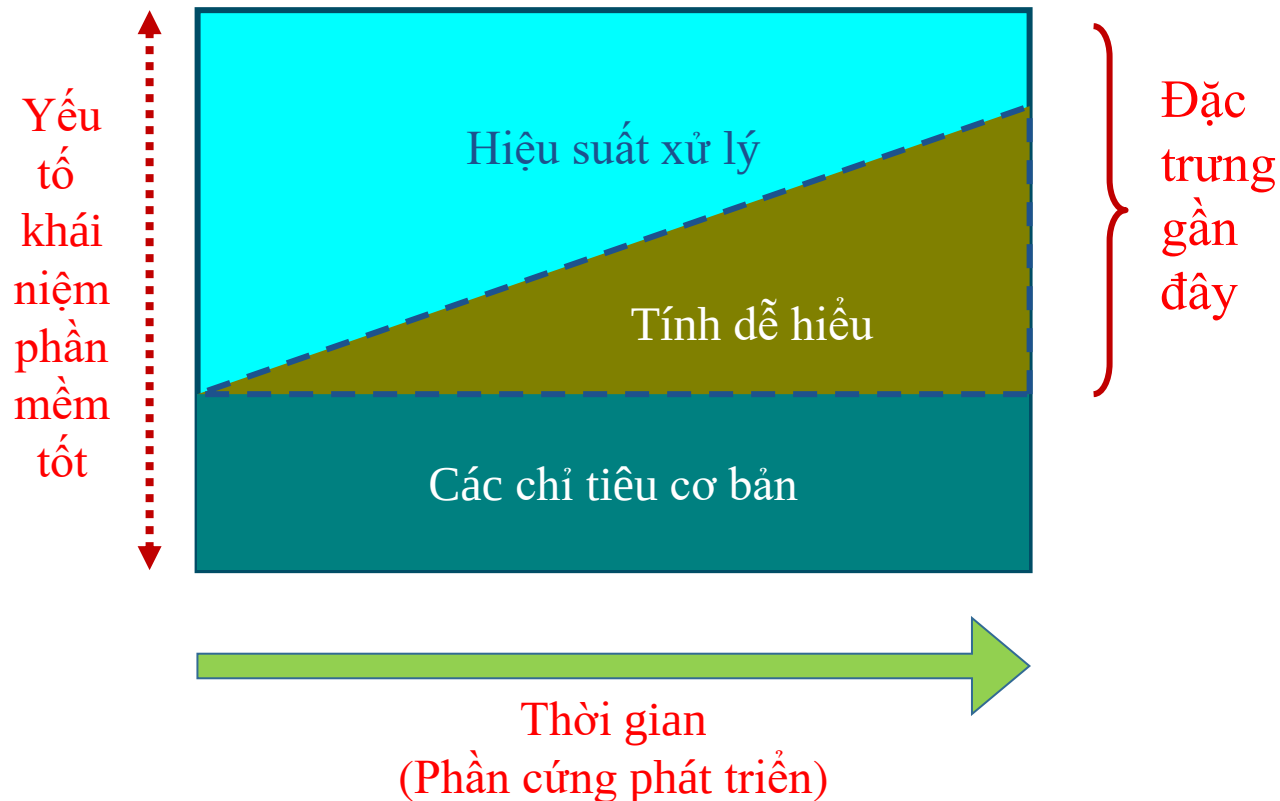
Thuộc tính của phần mềm

- **Khả năng bảo trì:** Nó có khả năng thực hành những tiến triển để thỏa mãn yêu cầu của khách hàng.
- **Khả năng tin cậy:** Khả năng tin cậy của phần mềm bao gồm một loạt các đặc tính như là độ tin cậy, an toàn, và bảo mật. Phần mềm tin cậy không thể tạo ra các thiệt hại vật chất hay kinh tế trong trường hợp hư hỏng.
- **Độ hữu hiệu:** Phần mềm không thể phí phạm các nguồn tài nguyên như là bộ nhớ và các chu kỳ vi xử lý.
- **Khả năng sử dụng:** Phần mềm nên có một giao diện tương đối dễ cho người dùng và có đầy đủ các hồ sơ về phần mềm (UX)



Tiêu chí của một phần mềm tốt

- Để đánh giá chất lượng phần mềm người ta đưa ra các tiêu chí có tính định tính





Các tiêu chí cơ bản

- Phản ánh đúng yêu cầu người dùng (tính hiệu quả effectiveness).
- Chứa ít lỗi tiềm tàng.
- Giá thành không vượt quá giá ước lượng ban đầu.
- Dễ vận hành, sử dụng.
- Tính an toàn và độ tin cậy cao.



Hiệu suất xử lý cao

- Hiệu suất thời gian tốt (efficiency)
 - Độ phức tạp tính toán thấp (time complexity)
 - Thời gian quay vòng ngắn (Turn Around Time TAT)
 - Thời gian hồi đáp nhanh (Response Time)
- Sử dụng tài nguyên hữu hiệu: CPU, RAM, HDD, Internet resources...



Tính dễ hiểu

- Kiến trúc và cấu trúc thiết kế dễ hiểu
- Dễ kiểm tra, kiểm thử, kiểm chứng
- Dễ bảo trì
- Có tài liệu (mô tả yêu cầu, điều kiện kiểm thử, vận hành, bảo trì, FAQ...) với chất lượng cao.

Tính dễ hiểu chỉ tiêu ngày càng quan trọng



Phân loại phần mềm

- Phân loại phần mềm thành 6 nhóm

Các hệ điều hành

Các chương trình biên dịch và thông dịch như PASCAL, C++, Visual Studio, FORTRAN, ADA...

Các chương trình hệ thống

Các hệ quản trị CSDL, tri thức và các chương trình mở rộng tương ứng như MS SQL Server, DB2...

Các chương trình ứng dụng

Các chương trình tiện ích, trò chơi.



Phân loại phần mềm

- Tùy theo tính ứng dụng của phần mềm mà người ta còn có thể phân loại:
 - Phần mềm hệ thống (System software)
 - Phần mềm thời gian thực(Real-Time SW)
 - Phần mềm nghiệp vụ(Business SW)
 - Phần mềm tính toán KH&KT (Eng.&Scie. SW)
 - Phần mềm nhúng (Embedded SW)
 - Phần mềm máy cá nhân(Personal C. SW)
 - Phần mềm dựa web (Web-base SW)
 - Phần mềm trí tuệ nhân tạo (AI SW)

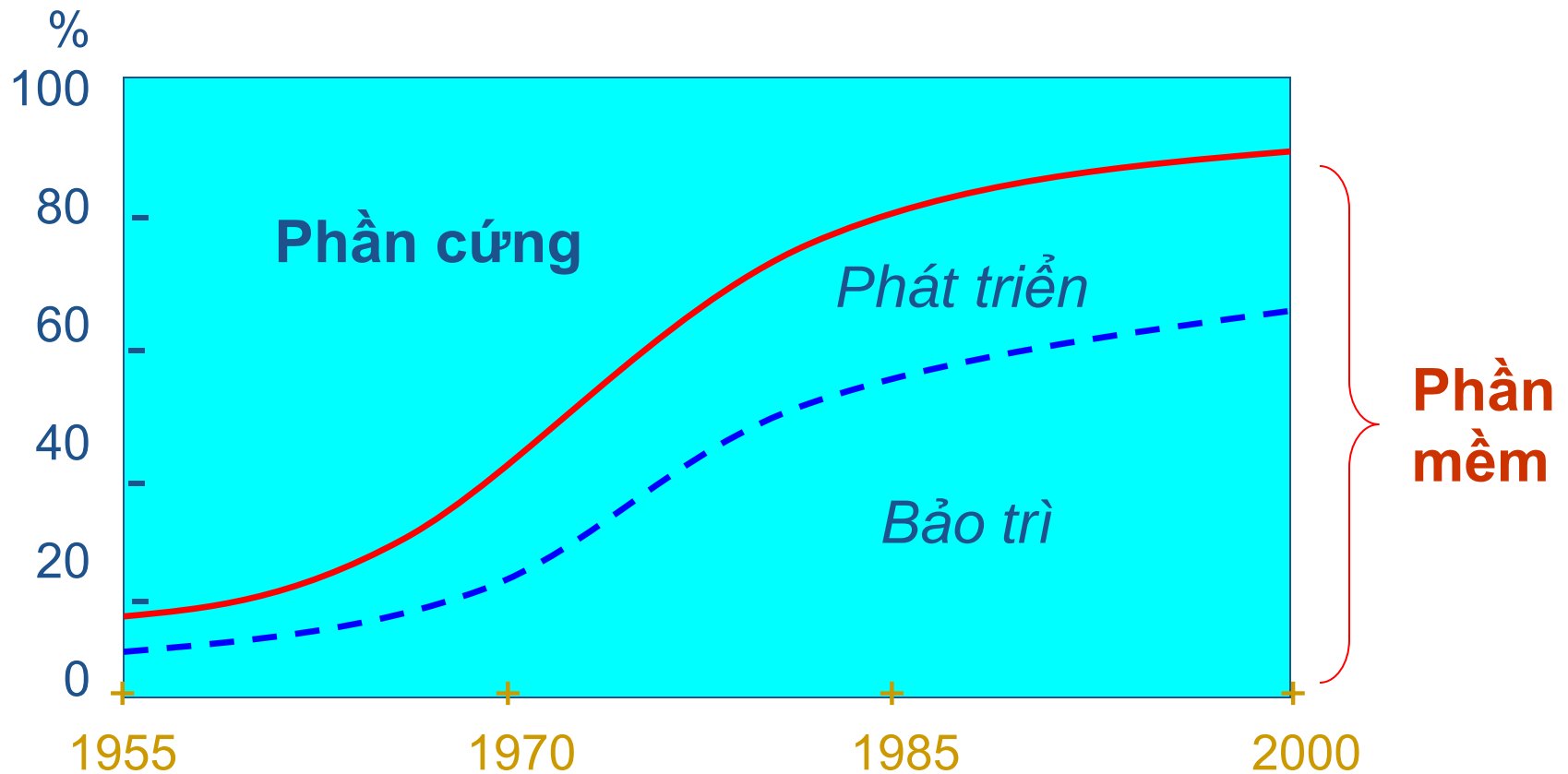


Phát triển phần mềm và các thách thức

- Một số yếu tố khi phát triển phần mềm:
 - Phần mềm càng lớn sẽ kéo theo phức tạp hóa và tăng chi phí phát triển.
 - Đổi vai trò giá thành phần cứng và phần mềm
 - Công sức cho bảo trì càng tăng thì chi phí cho backlog càng lớn.
 - Nhân lực chưa đáp ứng được nhu cầu phần mềm.
 - Những phiên hà của phần mềm gây ra những vấn đề xã hội.



So sánh chi phí cho phần cứng và phần mềm





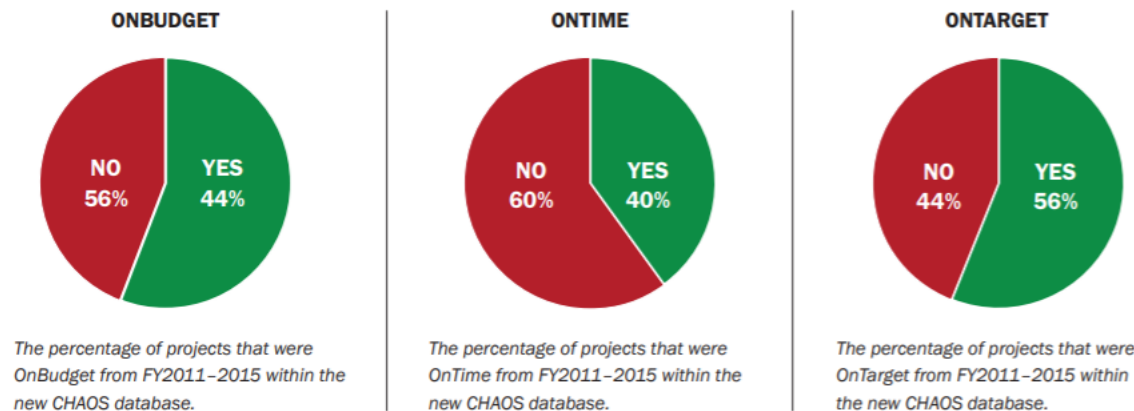
Khó khăn trong phát triển phần mềm

- Không có phương pháp mô tả rõ ràng định nghĩa yêu cầu người dùng (khách hàng) nên sau khi bàn giao sản phẩm dễ có trục trặc. Với những phần mềm quy mô lớn tư liệu đặc tả không thay đổi kịp thời với nhu cầu thay đổi của người dùng.
- Chuẩn về phần mềm tốt không thể đo được một cách định lượng nên không thể đánh giá được một hệ thống đúng đắn hay không do vậy độ tin cậy phần mềm sẽ giảm



Khó khăn trong phát triển phần mềm

- Mất quá nhiều thời gian và chi phí để xây dựng. Theo thống kê báo cáo của CHAOS năm 2015 cho tất cả các dự án:



TRADITIONAL RESOLUTION FOR ALL PROJECTS

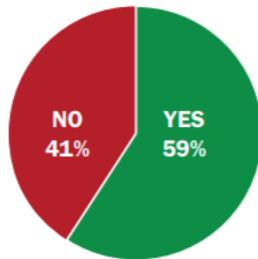
	2011	2012	2013	2014	2015
SUCCESSFUL	39%	37%	41%	36%	36%
CHALLENGED	39%	46%	40%	47%	45%
FAILED	22%	17%	19%	17%	19%

The Traditional resolution of all software projects from FY2011–2015 within the new CHAOS database.



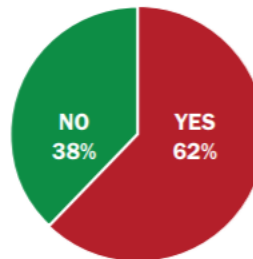
Báo cáo thống kê năm 2015

VALUABLE



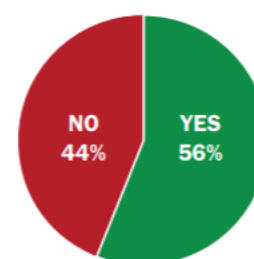
The percentage of projects considered valuable from FY2011-2015 within the new CHAOS database.

ONGOAL



The percentage of projects that were OnGoal from FY2011-2015 within the new CHAOS database.

SATISFACTORY



The percentage of projects considered satisfactory from FY2011-2015 within the new CHAOS database.

MODERN RESOLUTION FOR ALL PROJECTS

	2011	2012	2013	2014	2015
SUCCESSFUL	29%	27%	31%	28%	29%
CHALLENGED	49%	56%	50%	55%	52%
FAILED	22%	17%	19%	17%	19%

The Modern Resolution (OnTime, OnBudget, with a satisfactory result) of all software projects from FY2011-2015 within the new CHAOS database. Please note that for the rest of this report CHAOS Resolution will refer to the Modern Resolution definition not the Traditional



Khó khăn trong phát triển phần mềm

- Phần mềm phải khả dụng ở mọi nơi: Điện thoại di động, tích hợp vào tivi ...
- Phần mềm phải khả dụng trong hệ thống cần đảm bảo an toàn như máy năng lượng hạt nhân, máy bay, xe hơi...
- Khó để phát triển phần mềm cho các hệ thống phức tạp như hệ thống phân tán, hệ thống nhúng, hệ thống thời gian thực.
- Với những hệ thống có các yêu cầu bị thay đổi thì phần mềm nhanh chóng bị lạc hậu.

Một số kinh nghiệm khi phát triển phần mềm

- Cần phải có phương pháp luận thiết kế nhất quán và các chuẩn tư liệu về quy trình sản xuất phần mềm để nâng cao chất lượng phần mềm.
- Cần có chiến lược kiểm thử hợp lý để vừa có chi phí hợp lý vừa tránh nguy cơ vượt thời gian khi phát hiện lỗi ở kiểm thử giai đoạn cuối.

Một số kinh nghiệm khi phát triển phần mềm

- Nếu coi trọng việc lập trình hơn khâu thiết kế sẽ làm giảm chất lượng phần mềm.
- Không coi thường việc tái sử dụng phần mềm để tăng năng suất lao động.
- Việc bảo trì kéo dài sẽ làm giảm chất lượng tư liệu gây ảnh hưởng các khâu khác. Tuy chi phí cho bảo trì có xu hướng tăng nhưng nếu đầu tư nhân lực cho bảo trì lớn sẽ làm giảm hiệu suất lao động.

Một số kinh nghiệm khi phát triển phần mềm

- Quản lý dự án chặt chẽ kéo theo quản lý lịch trình rõ ràng đồng thời có tiêu chuẩn để ước lượng nhân lực rõ ràng sẽ tránh được nguy cơ kéo dài vượt thời hạn và kinh phí của dự án.



Nội dung

→ 2. Kỹ nghệ phần mềm

- Định nghĩa
- Đặc điểm về các pha trong vòng đời phần mềm
- Một số mô hình phát triển vòng đời phần mềm
 - Mô hình tuyến tính/vòng đời cổ điển/thác nước
 - Mô hình bản mẫu/chế thử
 - Mô hình gia tăng
 - Mô hình phát triển ứng dụng nhanh RAD
 - Mô hình xoắn ốc, xoắn ốc Win-Win
 - Mô hình linh hoạt Agile và quy trình Scrum



Kỹ nghệ phần mềm

- Kỹ nghệ phần mềm là việc thiết lập và sử dụng các nguyên lý công nghệ đúng đắn để thu được phần mềm một cách kinh tế vừa tin cậy, vừa làm việc hiệu quả trên các máy thực.
- Gồm 3 yếu tố chủ chốt:
 - Phương pháp: bao gồm các nhiệm vụ như lập kế hoạch và ước lượng dự án, phân tích yêu cầu hệ thống và phần mềm, thiết kế cấu trúc dữ liệu, kiến trúc chương trình, giao diện và thủ tục thuật toán, mã hóa, kiểm thử và bảo trì.



Kỹ nghệ phần mềm

- Công cụ: Là những cái hỗ trợ các phương pháp trên một cách tự động hoặc bán tự động. Phát triển phần mềm mà có công cụ hỗ trợ người ta gọi là CASE (Computer Aided Software Engineering)
- Thủ tục: Là chất keo dán các phương pháp và công cụ. Thủ tục xác định:
 - Trình tự các phương pháp sẽ được sử dụng
 - Sản phẩm cần bàn giao (tài liệu, báo cáo, bản mẫu...)
 - Cột mốc để người làm phần mềm nắm được tiến độ



Quy trình phát triển phần mềm

- Phát triển phần mềm gồm một tập các bước phải bao hàm đủ phương pháp, công cụ và thủ tục. Các bước này người ta gọi là khuôn cảnh phát triển phần mềm hay khuôn cảnh kỹ nghệ phần mềm hay các cách tiếp cận để phát triển phần mềm.
- Vòng đời phần mềm hay là quy trình phần mềm được phân chia thành các pha chính: phân tích, thiết kế, lập trình, kiểm thử, vận hành và bảo trì.



Đặc điểm về các pha trong vòng đời phần mềm

1. Pha xác định yêu cầu và thiết kế có vai trò quyết định đến chất lượng phần mềm, chiếm phần lớn công sức so với lập trình, kiểm thử và chuyển giao phần mềm.
2. Pha cụ thể hóa cấu trúc phần mềm phụ thuộc nhiều vào suy nghĩ trên xuống (top-down) và trừu tượng hóa, cũng như chi tiết hóa



Đặc điểm về các pha trong vòng đời phần mềm

3. Pha thiết kế, chế tạo thì theo trên xuống, pha kiểm thử thì dưới lên (bottom-up)
Trước khi chuyển sang pha kế tiếp phải đảm bảo pha hiện nay đã được kiểm thử không còn lỗi.
4. Cần có cơ chế kiểm tra chất lượng, xét duyệt giữa các pha nhằm đảm bảo không gây lỗi cho pha sau.

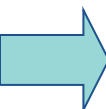


Đặc điểm về các pha trong vòng đời phần mềm

5. Tư liệu của mỗi pha không chỉ dùng cho pha sau, mà chính là đối tượng quan trọng cho kiểm tra và đảm bảo chất lượng của từng quy trình và của chính phần mềm.
6. Thao tác bảo trì phần mềm là việc xử lý quay vòng trở lại các pha trong vòng đời phần mềm nhằm biến đổi, sửa chữa, nâng cấp phần mềm.



Nội dung



– Một số mô hình phát triển vòng đời phần mềm

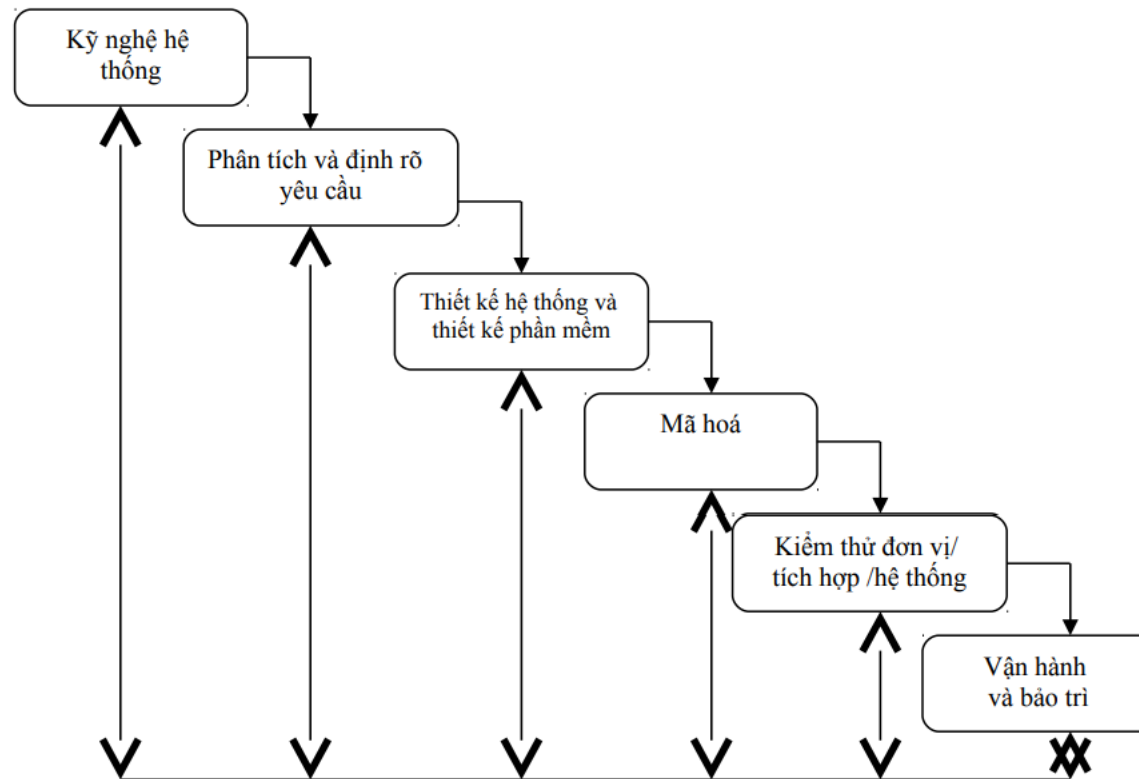
- Mô hình tuyến tính/vòng đời cổ điển/thác nước
- Mô hình bản mẫu/chế thử
- Mô hình gia tăng
- Mô hình phát triển ứng dụng nhanh RAD
- Mô hình xoắn ốc, xoắn ốc Win-Win
- Mô hình linh hoạt Agile và quy trình Scrum



Mô hình vòng đời cổ điển

(Classic life cycle / waterfall model)

- Hoạt động như sau:





Mô hình vòng đời cổ điển

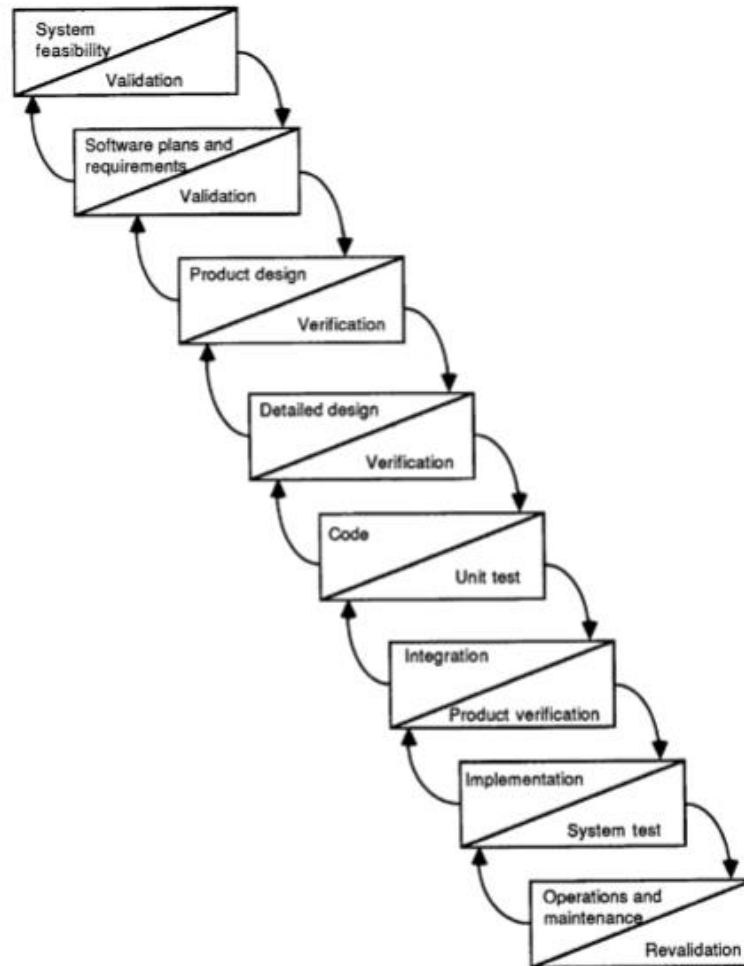


Figure 1. The waterfall model of the software life cycle.



Mô hình vòng đời cổ điển

- Theo mô hình này các pha được thực hiện trình tự: bắt đầu ở mức hệ thống tiến dần xuống phân tích, thiết kế, mã hóa, kiểm thử và bảo trì.
- Phát triển hệ thống: Công việc phát triển hệ thống là công việc thiết lập ra các yêu cầu cho hệ thống, ánh xạ một số tập con các yêu cầu này sang phần mềm.



Mô hình vòng đời cổ điển

- Thiết kế là tập trung vào 4 thuộc tính phân biệt của chương trình: Cấu trúc dữ liệu, kiến trúc phần mềm, chi tiết thủ tục, đặc trưng giao diện.
- Mã hóa là công việc lập trình viết code.
- Kiểm thử: Phần kiểm thử không chỉ bắt đầu sau khi chúng ta đã viết mã, mà ở đây chúng ta có nhiều loại kiểm thử.
- Vận hành bảo trì: Đáp ứng những thay đổi, nâng cấp phần mềm đã phát triển do sự thay đổi của môi trường, nhu cầu



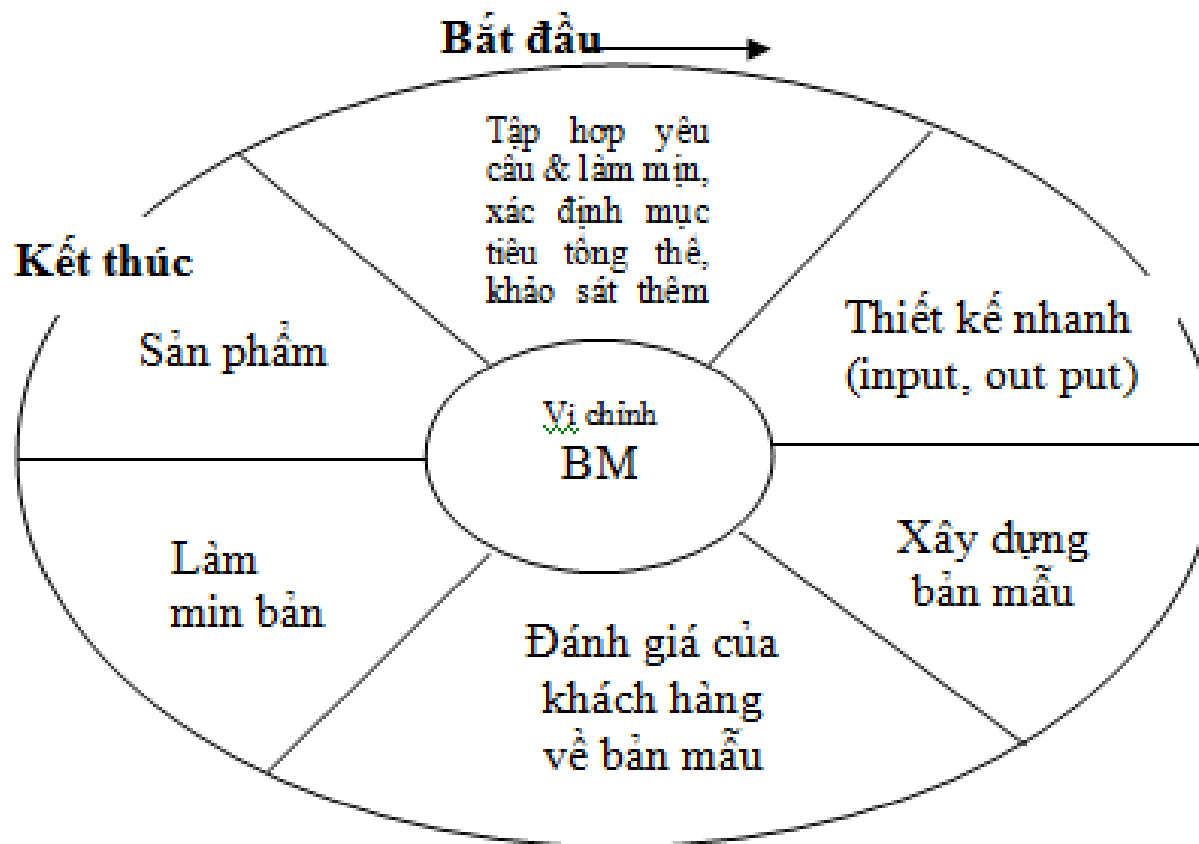
Mô hình vòng đời cổ điển

- Điểm yếu:
 - Thực tế các dự án ít khi tuân theo dòng tuần tự của mô hình, mà thường có lặp lại (như mô hình của Boehm)
 - Khách hàng ít khi tuyên bố rõ ràng khi nào xong hết các yêu cầu
 - Khách hàng phải có lòng kiên nhẫn chờ đợi thời gian nhất định mới có sản phẩm. Nếu phát hiện ra lỗi nặng thì là một thảm họa!



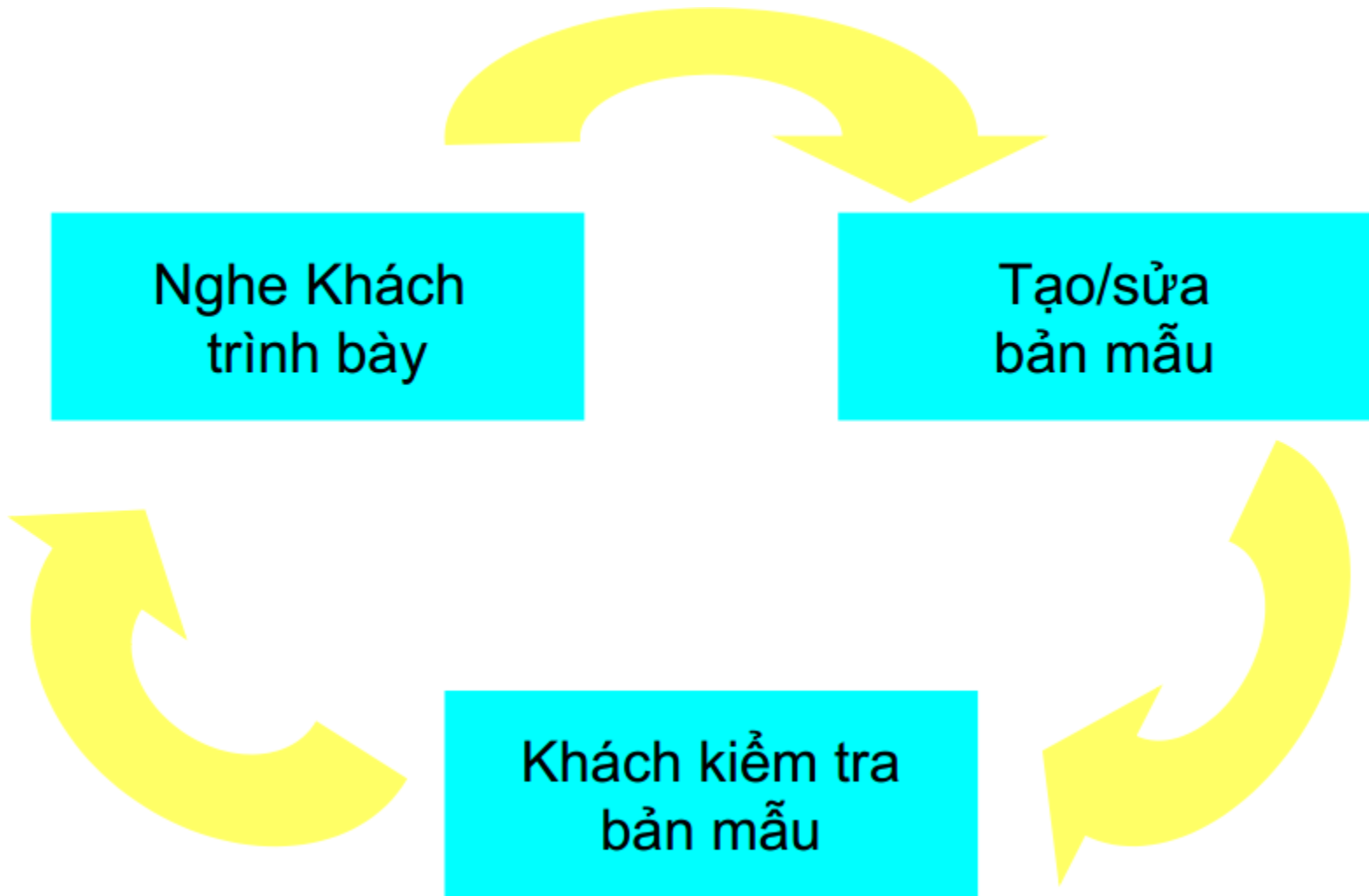
Mô hình chế thử

(Mô hình bản mẫu/prototyping model)





Mô hình chế thử





Mô hình chế thử

- Dùng mô hình bản mẫu khi cần tạo như một “hệ sơ khai” để thu thập yêu cầu người dùng qua các thiết kế nhanh khi:
 - Thông thường khách hàng đã xác định được mục tiêu tổng quát của phần mềm, nhưng chưa xác định được cái vào, xử lý hay yêu cầu cái ra; người phân tích chưa hiểu rõ nhu cầu của khách hàng.
 - Hay khi người phát triển có thể không chắc về tính hiệu quả của một thuật toán hay giải pháp, việc thích nghi hệ điều hành hay dạng giao diện người máy (HCI – *Human Computer Interface*) cần có.



Mô hình chế thử

- Mô hình bản mẫu có thể ở trong 3 dạng:
 1. Bản mẫu trên giấy hay mô hình dựa trên máy PC mô tả giao diện người máy dưới dạng làm cho người dùng hiểu được cách các tương tác xuất hiện.
 2. Bản mẫu làm việc cài đặt một tập con các chức năng của phần mềm mong muốn
 3. Một chương trình đã có thực hiện một phần hay tất cả các chức năng mong muốn nhưng cần phải cải tiến thêm các tính năng khác tùy theo khả năng phát triển.



Mô hình chế thử

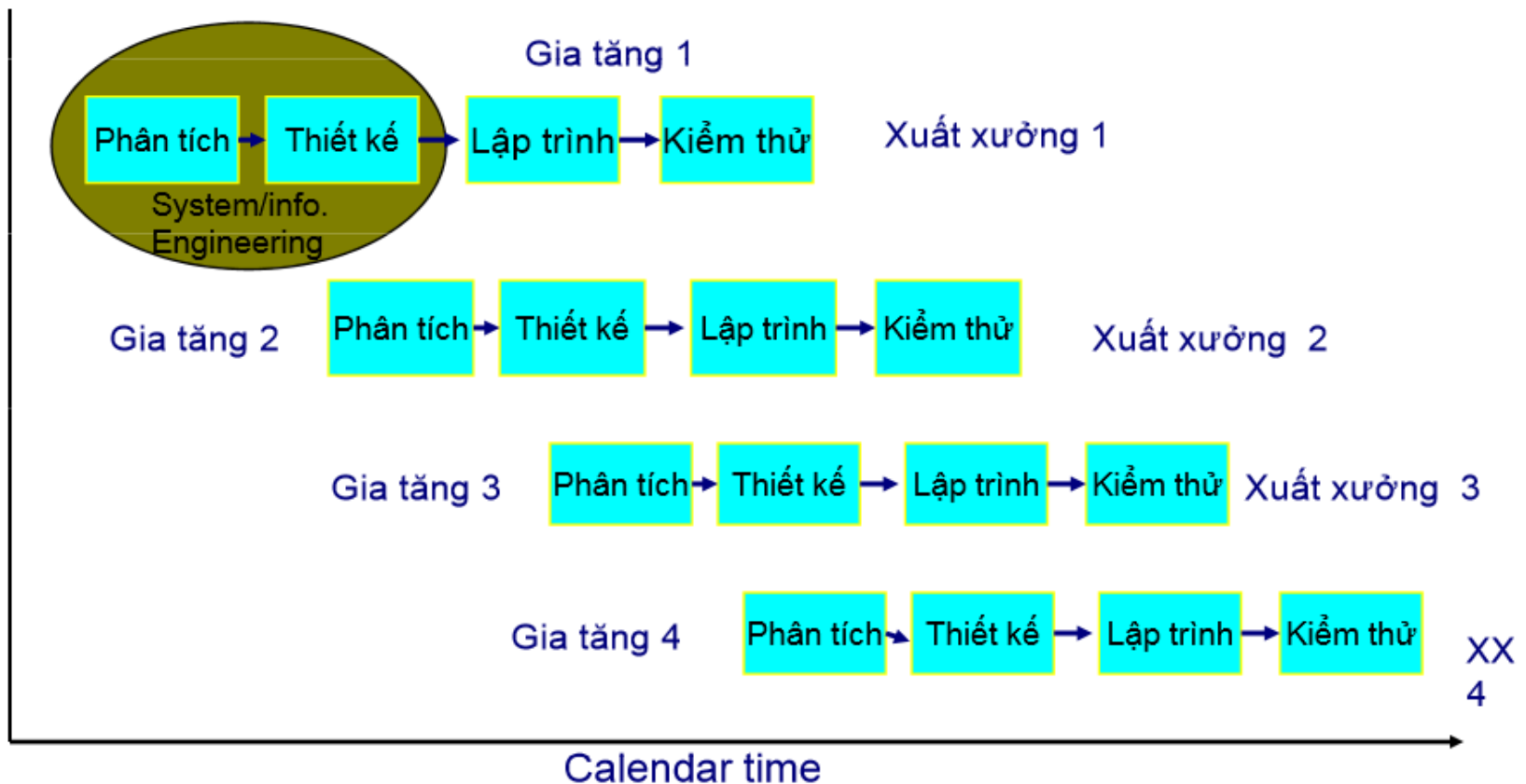
- Ví dụ một mô hình dựa trên giấy mô phỏng tương tác thi trắc nghiệm trên di động





Mô hình gia tăng

(The incremental model)





Ví dụ



- Trong ví dụ trên chúng ta phải làm việc gia tăng thêm các miếng ghép cho đến khi bức tranh ý tưởng được hoàn thành.



Mô hình gia tăng

- Là sự kết hợp giữa mô hình tuần tự và ý tưởng lặp lại của chế bản mẫu.
- Sản phẩm lỗi với những yêu cầu cơ bản nhất của hệ thống được phát triển trước. Các chức năng với những yêu cầu khác được phát triển thêm sau (gia tăng)
- Lặp lại quy trình để hoàn thiện dần



Mô hình gia tăng

- Mô hình này được đề xuất dựa trên ý tưởng thay vì phải xây dựng và chuyển giao hệ thống một lần thì sẽ chia thành nhiều giai đoạn tăng dần. mỗi giai đoạn là một phần kết quả chức năng được yêu cầu.
- Các yêu cầu của khách hàng được đánh thứ tự ưu tiên. Yêu cầu nào được đánh thứ tự ưu tiên càng cao thì càng ở trong những giai đoạn phát triển sớm và được kiểm tra kỹ lưỡng.



Mô hình gia tăng

- Sử dụng mô hình gia tăng khi nào?
 - Khi yêu cầu của dự án là rõ ràng, đầy đủ, và nắm rõ được các yêu cầu của dự án.
 - Khi sớm cần có một nguyên mẫu phần mềm để quảng bá, giới thiệu hoặc thử nghiệm.
 - Khi một công nghệ mới được áp dụng.
 - Tài nguyên và kỹ năng chuyên môn luôn sẵn sàng.
 - Khi có một tính năng hay các mục tiêu có nguy cơ lỗi cao.



Mô hình gia tăng

- Ưu điểm:
 - Có thể sớm tạo ra nguyên mẫu của sản phẩm
 - Độ linh hoạt cao hơn và khi thay đổi yêu cầu dự án thì chi phí sẽ ít hơn nhiều, vì những thay đổi thuộc về module nào thì module đó sẽ thay đổi mà các module khác không hề bị ảnh hưởng.
 - Làm cho việc test nhẹ nhàng hơn, những module đơn giản thì test cũng đơn giản, sớm kết thúc.
 - Giảm chi phí cho lần đầu giao sản phẩm.
 - Dễ dàng quản lý các rủi ro có thể phát sinh.



Mô hình gia tăng

- Nhược điểm:
 - Cần phải có những khả năng thiết kế tốt và phương pháp tốt.
 - Cần hiểu rõ được yêu cầu và biết cách phân chia nó ra như thế nào cho hợp lý.
 - Chi phí để phát triển theo phương pháp này là rất cao, cao hơn hẳn mô hình thác nước.

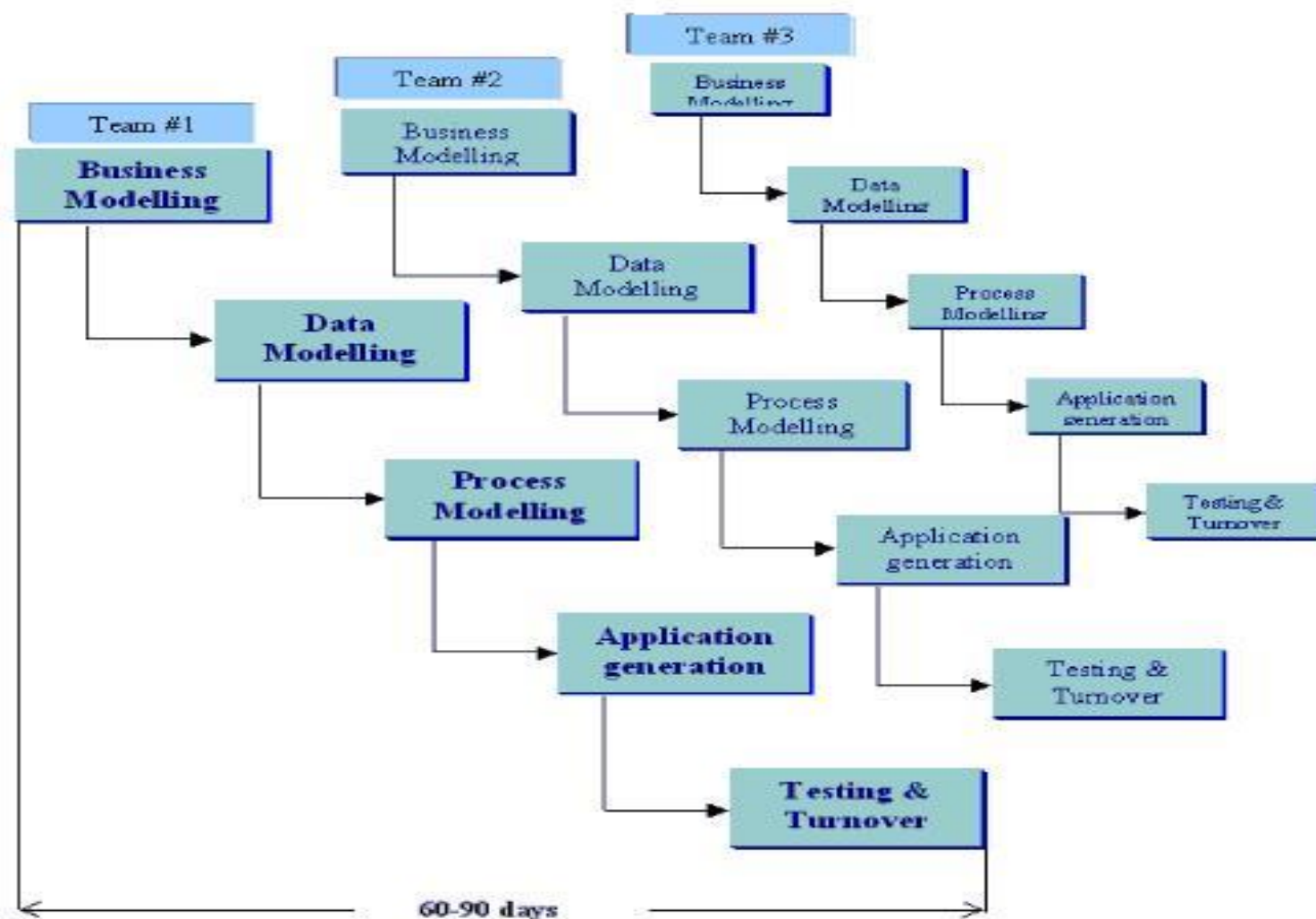


Mô hình RAD

- Là một dạng khác của mô hình tiến trình gia tăng(Incremental Process). Nó dựa trên việc tạo mẫu và phát triển lặp nhấn mạnh đến chu kỳ phát triển ngắn (60-90 ngày).
- Nó tập trung vào việc thu thập các yêu cầu của khách hàng, thử nghiệm sớm nguyên mẫu bằng cách sử dụng khái niệm lặp, tái sử dụng nguyên mẫu hiện có(thành phần), tích hợp liên tục và chuyển giao nhanh sản phẩm.



Mô hình RAD





Mô hình RAD

- Mỗi nguyên mẫu là một mô hình làm việc có chức năng tương đương với một thành phần của sản phẩm.
- Các môđun chức năng được phát triển song song như các nguyên mẫu, nghĩa là các nhóm nhỏ(team) hoạt động độc lập như các dự án nhỏ.



Mô hình RAD

- Mỗi nguyên mẫu đều được phát triển với chu trình phát triển lặp qua các pha: phân tích(Business Modeling), thiết kế(Data Modeling & Process Modeling), cài đặt (Application Generation), kiểm thử (Testing and turnover).
- Điều quan trọng nhất cho mô hình này thành công đó là nguyên mẫu được phát triển có thể tái sử dụng được.



Mô hình RAD

- Business modeling: Xác định các luồng thông tin giữa các chức năng khác nhau trong cùng một nguyên mẫu. Bao gồm:
 - Phân tích nghiệp vụ để tìm ra luồng thông tin chính cho các chức năng.
 - Làm thế nào để thu được luồng thông tin?
 - Làm thế nào và khi nào xử lý thông tin?
 - Những yếu tố nào thúc đẩy thành công luồng thông tin?



Mô hình RAD

- Data Modeling:
 - Luồng thông tin được phân tích ra trong pha trước sẽ được xem xét và phân tích để tìm ra các bộ đối tượng quan trọng cho các chức năng.
 - Xác định để tìm ra và định nghĩa các thuộc tính của tập dữ liệu.
 - Thiết lập và định nghĩa các mối quan hệ giữa các đối tượng dữ liệu trong mô hình nghiệp vụ.



Mô hình RAD

- Process Modeling:
 - Các đối tượng dữ liệu được tìm ra ở mô hình dữ liệu sẽ được xử lý chuyển đổi để thực hiện đúng theo luồng thông tin nghiệp vụ đã được xác định trong mô hình nghiệp vụ.
 - Các quá trình xử lý dữ liệu bao gồm: Thêm xóa, truy xuất hoặc sửa đổi các đối tượng dữ liệu.
- Application Generation: Công đoạn cài đặt ra nguyên mẫu.



Mô hình RAD

- Testing and Turnover:
 - Thời gian kiểm thử tổng thể của mô hình RAD được giảm thiểu do đã được kiểm thử độc lập trong suốt mỗi lần lặp của mỗi nguyên mẫu.
 - Tuy nhiên luồng dữ liệu và giao diện giữa các nguyên mẫu thành phần cần phải được kiểm thử kỹ lưỡng khi kết hợp tạo ra sản phẩm hoàn chỉnh.
 - Vì các nguyên mẫu đều đã được kiểm thử nên nó làm giảm thiểu nguy cơ rủi ro của dự án.



Mô hình RAD

- So sánh với các mô hình truyền thống:
 - Điểm yếu của mô hình truyền thống là sự cứng nhắc với việc nhấn mạnh phân tích yêu cầu và thu thập dữ liệu trước khi đưa ra sản phẩm nguyên mẫu, vì vậy nó gây ra áp lực cho khách hàng khi phải ký kết hợp đồng trong khi họ chưa cảm thấy được sản phẩm.
 - Việc thay đổi theo yêu cầu của khách hàng trong MHTT là khá cứng nhắc hoặc có thể không khả thi. Trong khi đó mô hình RAD việc thay đổi rất linh hoạt nhờ việc tạo nguyên mẫu và sự độc lập môđun giữa các thành phần.



Mô hình RAD

- Khi nào nên áp dụng mô hình RAD:
 - Chỉ sử dụng với các sản phẩm phần mềm có tính môđun hóa để có thể chuyển giao theo cách gia tăng.
 - Nên sử dụng khi có sẵn nhân lực và các nhà thiết kế có trình độ cao, các nhân lực chuyên gia về lĩnh vực nghiệp vụ của sản phẩm.
 - Nên sử dụng khi có yêu cầu thay đổi của khách hàng trong suốt dự án và thời gian hoàn thành ngắn trong 2-3 tháng.



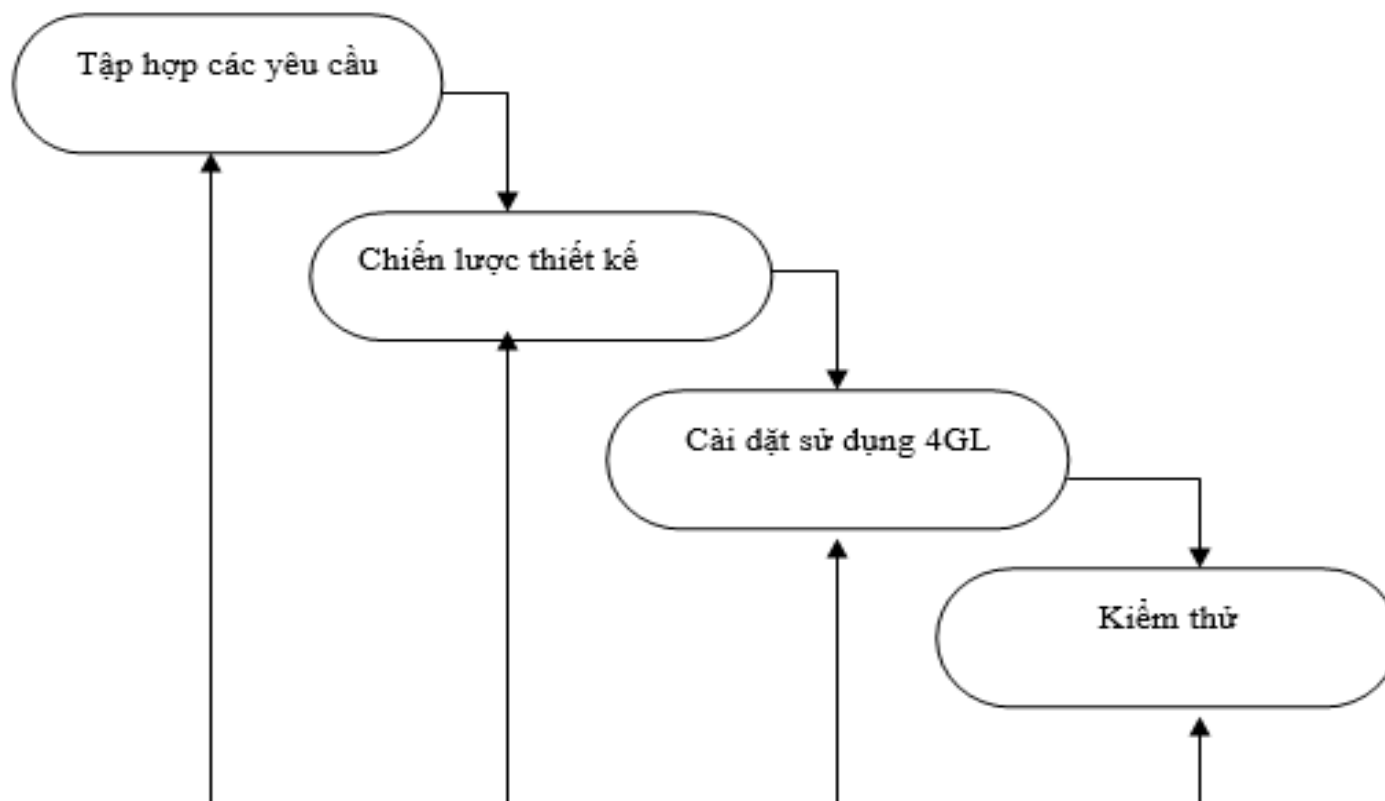
Mô hình RAD kết hợp với 4GT

- Kỹ thuật thể hệ thứ 4:
 - Bao gồm một phạm vi rộng các công cụ phần mềm có khả năng tự động sinh ra mã code chương trình gốc theo nhu cầu của người phát triển gần với ngôn ngữ tự nhiên.
 - Môi trường phát triển PM hỗ trợ 4GT gồm trong số các công cụ: Ngôn ngữ phi thủ tục để truy vấn DL, bộ sinh báo cáo, bộ thao tác DL, bộ tương tác và xác định màn hình, bộ sinh chương trình, khả năng đồ họa mức cao, khả năng làm trang tính.



Mô hình RAD kết hợp với 4GT

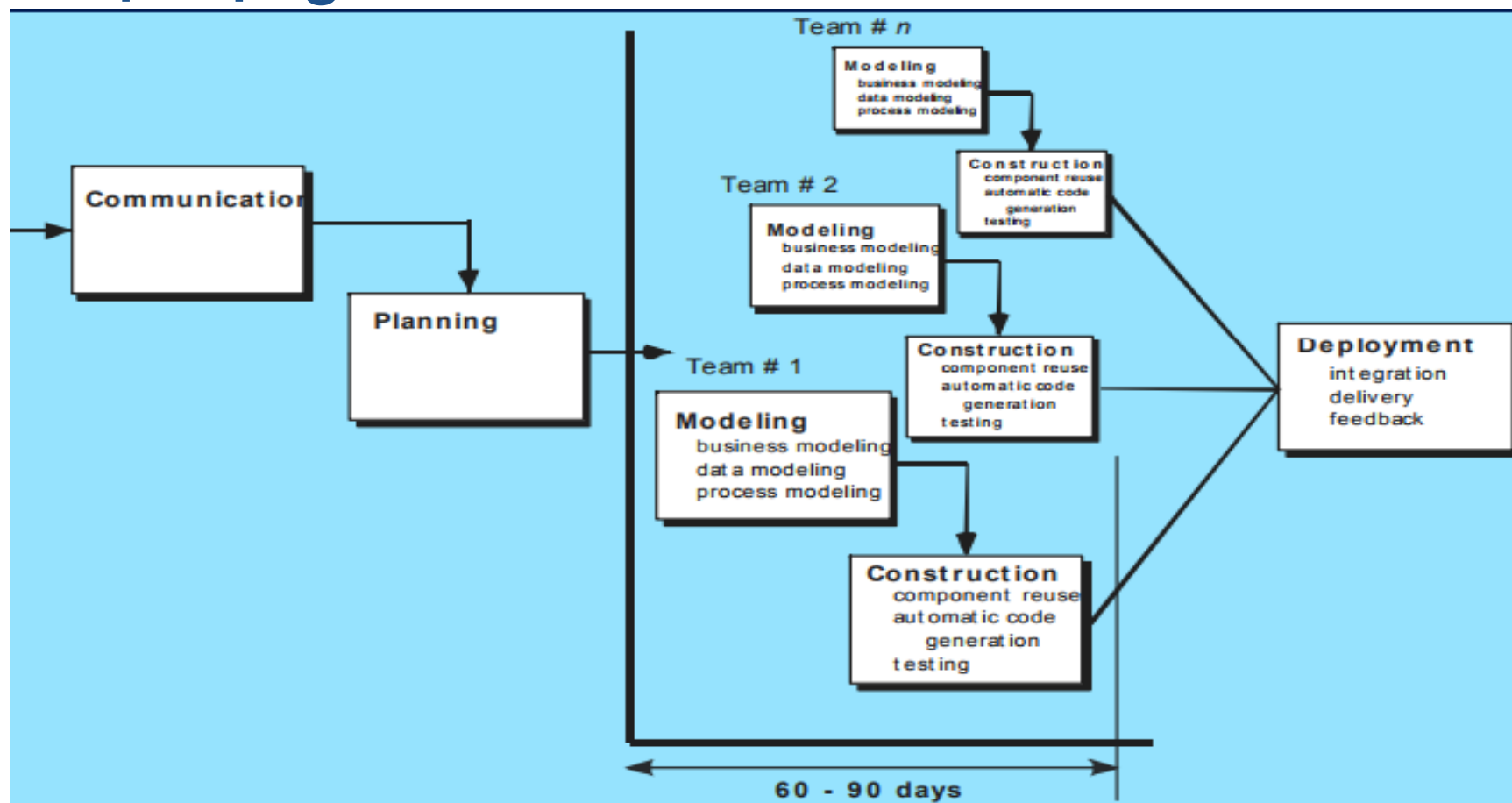
- Các kỹ thuật thể hệ thứ tư:





Mô hình RAD kết hợp với 4GT

- Là mô hình RAD kết hợp với các công cụ tự động sinh mã code.





Mô hình RAD kết hợp với 4GT

- Hiện nay trên thị trường có rất nhiều phần mềm hỗ trợ mô hình RAD kết hợp với 4GT. Chi tiết hơn về các sản phẩm này:

<https://www.g2crowd.com/categories/rapid-application-development-rad>



Mô hình xoắn ốc

- Là quy trình phát triển định hướng rủi ro cho các dự án phần mềm, mô hình xoắn ốc đưa ra cách áp dụng các yếu tố của một hoặc nhiều mô hình xử lý, chẳng hạn như mô hình thác nước, mô hình bản mẫu tiến hóa.
- Mô hình này lần đầu được Barry Boehm đưa ra. Mỗi cấp độ trong xoắn ốc liên quan đến việc lập kế hoạch, phân tích, phát hiện rủi ro, hoàn thiện hệ thống và tạo mẫu thêm.



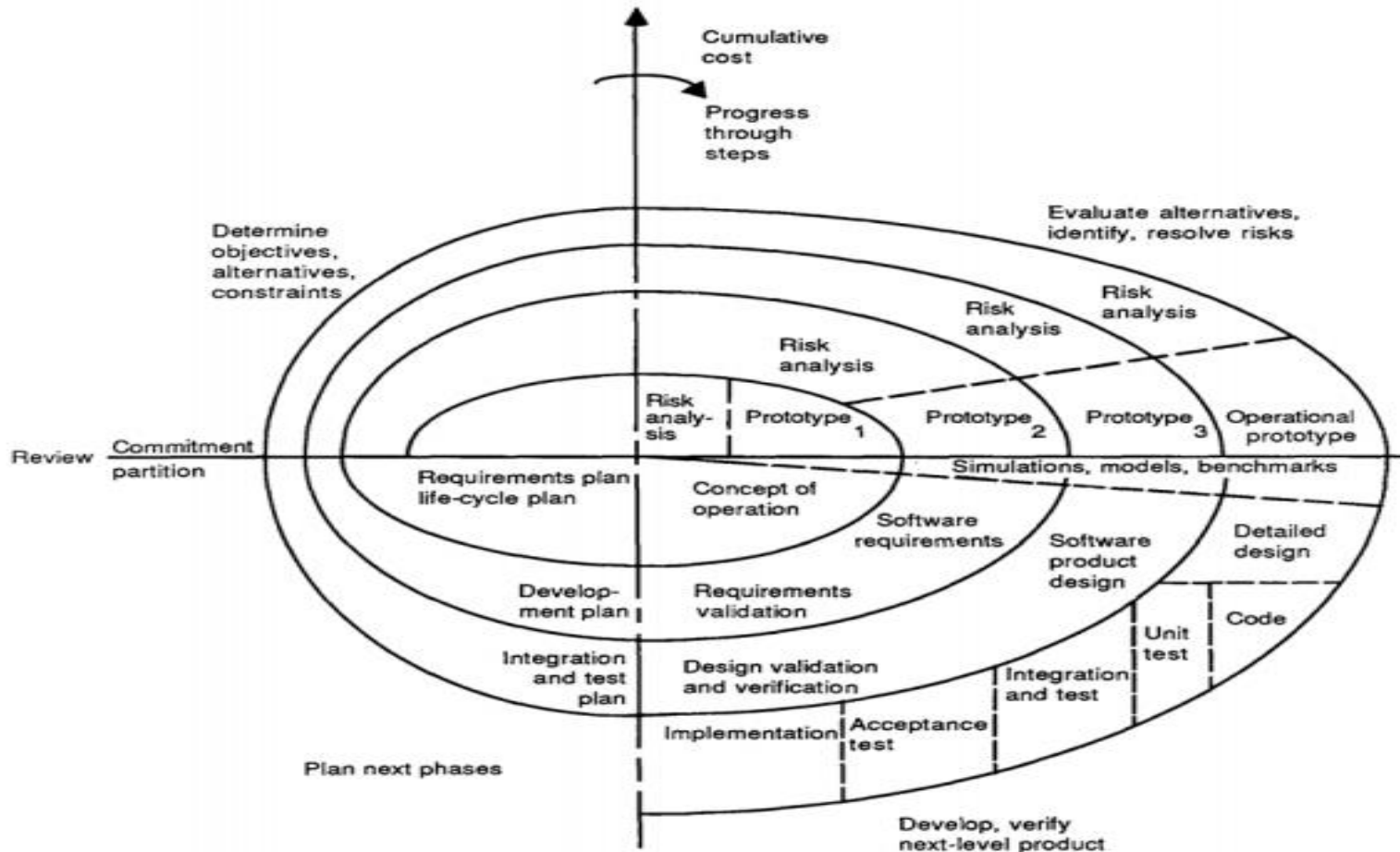
Mô hình xoắn ốc

- Về bản chất, nó mô tả sự phát triển của phần mềm qua các giai đoạn tiến hóa, mỗi giai đoạn được coi như một mô hình thác nước, bổ sung vào giai đoạn thực hiện PM mẫu (prototype) ngay sau khi xác định YC nhằm mục tiêu phát hiện nhanh các sai sót về yêu cầu.
- Trong mỗi vòng lặp của đường xoắn ốc, khách hàng đánh giá và đề nghị được sửa đổi. Thêm vào đó với mỗi vòng lặp, một phân tích rủi ro được thực hiện cho kết quả là tiếp tục dự án hay dừng lại.



Mô hình xoắn ốc

Mô hình xoắn ốc (spiral model)





Các hoạt động chính của mỗi bước lập trong quy trình

1. *Lập kế hoạch*: xác định mục tiêu, giải pháp và ràng buộc
2. *Phân tích rủi ro*: phân tích các phương án và xác định, giải quyết rủi ro.
3. *Kỹ nghệ*: phát triển sản phẩm “mức tiếp”
4. *Đánh giá của khách hàng*: Đánh giá kết quả của kỹ nghệ.



Phân tích rủi ro

1. Hãy xem xét đến các điều kiện quan trọng nhất của các yếu tố liên quan từ đó xác định rủi ro và đặt độ ưu tiên cho các rủi ro.
2. Thiết lập các bản mẫu cho các rủi ro đã xác định và bắt đầu với cái có độ ưu tiên cao nhất.
3. Nếu một rủi ro giải quyết thành công, đánh giá kết quả và lập kế hoạch cho vòng tiếp theo
4. Nếu một rủi ro nào không giải quyết được, kết thúc dự án ngay lập tức



Spiral Model Template

- Phân tích những rủi ro và khả năng giải quyết thường là xây dựng bản mẫu.
- Để xác định rủi ro của mỗi giai đoạn trong mỗi xoắn ốc, Boehm sử dụng mẫu "Spiral Model Template".
- Được [Barry Boehm](#) lần đầu đưa ra trong bài báo năm 1968 với tựa đề "A Spiral Model of Software Development and Enhancement " [\(1\)](#)



Spiral Model Template

Template	Explanation	Example Phase
Objectives	The goals of the software project	Significantly improve software quality
Constraints	Limitations which the project must meet	Within three years Without large-scale capital investment Without radical change to company standards
Alternatives	Possible ways to achieve the objectives	Reuse existing certified software Introduce formal specification and verification Invest in testing and validation tools
Risks	Potential risks for this phase	No cost effective quality improvement possible Quality improvements may increase costs excessively New methods might cause existing staff to leave
Risk Resolution	Strategies for reducing the risks	Literature survey, Pilot project, Survey of potential reusable components, Assessment of available tool support, Staff training and motivation seminars
Results	Results of applying risk resolution strategies	Experience of formal methods is limited - very hard to quantify improvements Limited tool support available for company standard development system Reusable components available but little reuse tool support
Plans	Development plans for the next phase	Explore reuse option in more detail Develop prototype reuse support tools Explore component certification scheme
Commitment	Resources needed to achieve the plans	Fund further 18-month study phase



Kỹ nghệ

- Phát triển và triển khai
 - Dựa trên việc lập kế hoạch và phân tích rủi ro để từ đó phát triển hệ thống đồng thời phải kiểm tra lại. Giai đoạn này ta nên sử dụng mô hình thác nước để phát triển dự án.
- Lập kế hoạch cho pha tiếp theo
 - Chúng ta xem xét tiến độ và đánh giá thông qua các thông số đã đưa ra ở bước lập kế hoạch. Từ đó, tiếp tục triển khai giải quyết các vấn đề còn lại với qui trình được lặp lại tương tự.



Đặc điểm của mô hình

- Mô hình xoắn ốc cung cấp cách thức làm phần mềm bằng cách đưa ra các phiên bản tăng dần:
 - Đây không phải là bổ sung thêm các thành phần mới như mô hình gia tăng.
 - Đây là sự tiến hóa: Cũng các đặc trưng ấy nhưng được làm mịn hơn, chi tiết hơn, cũng như nêu ra được các rủi ro mới cần giải quyết
 - Phiên bản sau cùng chính là phần mềm hoàn chỉnh có thể chuyển giao cho khách hàng sử dụng.



Những quan niệm sai lầm nguy hiểm

- Mô hình xoắn ốc đơn giản chỉ là một chuỗi sự phát triển của mô hình thác nước.
- Tất cả các hoạt động dự án theo một trình tự xoắn ốc đơn.
- Mọi hoạt động trong sơ đồ phải được thực hiện, và theo thứ tự hiển thị.



Ưu điểm của mô hình xoắn ốc

- Là mô hình hội tụ các tính năng tốt và khắc phục các yếu điểm của nhiều mô hình phát triển khác gặp phải.
- Giám sát dự án dễ dàng và hiệu quả.
- Rất phù hợp với dự án lớn cần phát triển nhiều phiên bản, các hệ thống có yêu cầu xác định chưa rõ ràng, có nguy cơ cao và giảm thiểu rủi ro, đối phó với những thay đổi trong quá trình thực hiện dự án.
- Dự đoán về thời hạn và chi phí sát với thực tế.



Nhược điểm của mô hình xoắn ốc

- Phân tích rủi ro khá tốn kém, chủ yếu áp dụng cho dự án lớn, có tiềm lực về tài chính
- Yêu cầu thay đổi thời xuyên dẫn đến lặp vô hạn, phức tạp, cần có đội ngũ chuyên gia về phân tích rủi ro
- Chưa được áp dụng rộng rãi như mô hình thác nước, nguyên mẫu.



Khi nào nên sử dụng MH xoắn ốc





Mô hình xoắn ốc Win-Win

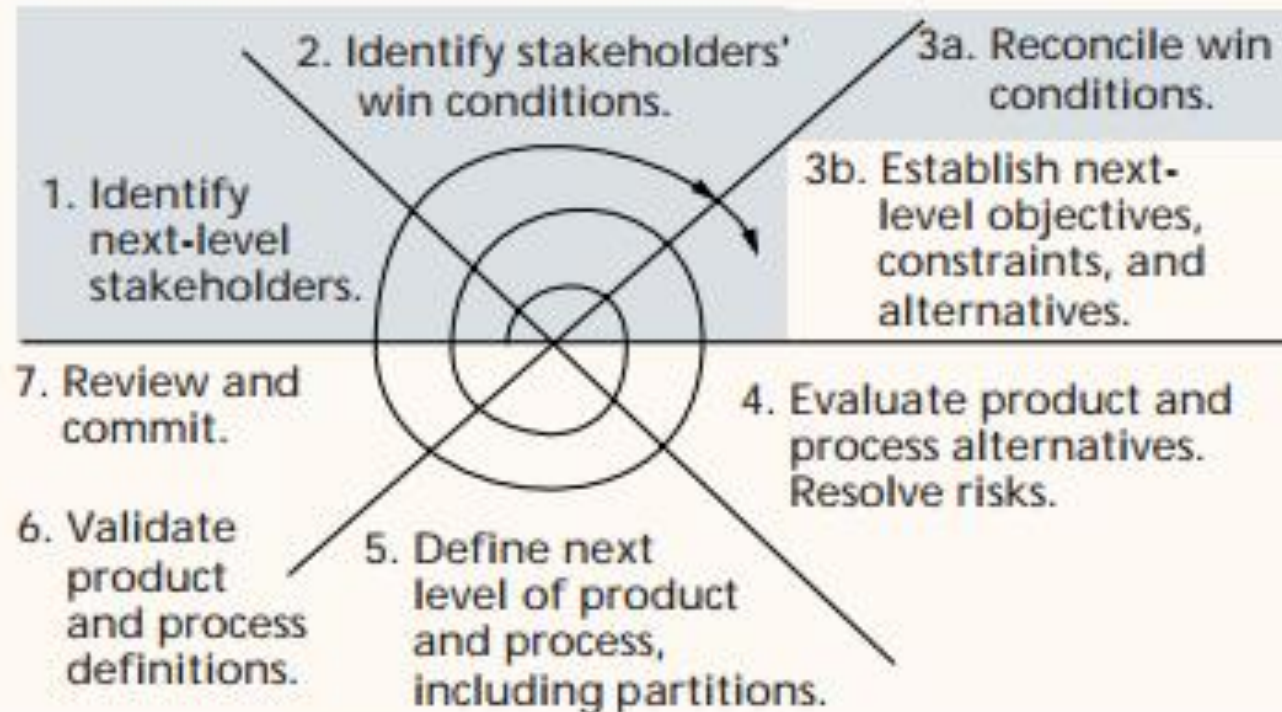


Figure A. How the WinWin spiral model differs from the original spiral model. The new model adds front-end activities (blue) that show where objectives, constraints, and alternatives come from. This lets users more clearly identify the rationale involved in negotiating win conditions for the product.



Mô hình xoắn ốc Win-Win

- Mô hình xoắn ốc Win-Win được đưa ra bởi Barry Boehm là sự mở rộng của mô hình xoắn ốc, nhằm thỏa hiệp giữa người phát triển và khách hàng, cả hai cùng thắng.
- Các giai đoạn trong mô hình này giống như mô hình xoắn ốc, chỉ khác ở chỗ thời điểm xác định yêu cầu, đội phát triển và khách hàng cùng thảo luận và đàm phán. (Là phần bổ sung màu xanh)



Mô hình linh hoạt Agile

- Là cách tiếp cận dựa trên nguyên tắc phát triển phân đoạn lặp (iterative) và tăng trưởng (incremental) do đó nhu cầu giải pháp tiến hóa thông qua sự hợp tác giữa các nhóm tự tổ chức và liên chức năng.
- Agile thường sử dụng cách lập kế hoạch thích ứng (adaptive planning), việc phát triển và chuyển giao theo hướng tiến hóa; sử dụng các khung thời gian ngắn và linh hoạt để dễ dàng phản hồi lại với các thay đổi trong quá trình phát triển. Yêu cầu tính tương tác với khách hàng.

Tuyên ngôn Phát triển phần mềm linh hoạt

Cá nhân và sự tương tác hơn là quy trình và công cụ;

Phần mềm chạy tốt hơn là tài liệu đầy đủ;

Cộng tác với khách hàng hơn là đàm phán hợp đồng;

Phản hồi với các thay đổi hơn là bám sát kế hoạch.

Mặc dù các điều bên phải vẫn còn giá trị, nhưng chúng tôi đánh giá cao hơn các mục ở bên trái.



Agile được vận dụng trong thực tế theo các nguyên lý

1. Ưu tiên cao nhất của chúng tôi là thỏa mãn khách hàng thông qua việc chuyển giao sớm và liên tục các phần mềm có giá trị.
2. Chào đón việc thay đổi yêu cầu, thậm chí rất muộn trong quá trình phát triển. Các quy trình linh hoạt tận dụng sự thay đổi cho các lợi thế cạnh tranh của khách hàng.
3. Thường xuyên chuyển giao phần mềm chạy tốt tới khách hàng, từ vài tuần đến vài tháng, ưu tiên cho các khoảng thời gian ngắn hơn.



Agile được vận dụng trong thực tế theo các nguyên lý

4. Nhà kinh doanh và nhà phát triển phải làm việc cùng nhau hàng ngày trong suốt dự án.
5. Xây dựng các dự án xung quanh những cá nhân có động lực. Cung cấp cho họ môi trường và sự hỗ trợ cần thiết, và tin tưởng họ để hoàn thành công việc.
6. Phương pháp hiệu quả nhất để truyền đạt thông tin tới nhóm phát triển và trong nội bộ nhóm phát triển là hội thoại trực tiếp.
7. Phần mềm chạy tốt là thước đo chính của tiến độ.



Agile được vận dụng trong thực tế theo các nguyên lý

8. Các quy trình linh hoạt thúc đẩy phát triển bền vững. Các nhà tài trợ, nhà phát triển, và người dùng có thể duy trì một nhịp độ liên tục không giới hạn.
9. Liên tục quan tâm đến các kỹ thuật và thiết kế tốt để gia tăng sự linh hoạt.
10. Sự đơn giản – nghệ thuật tối đa hóa lượng công việc chưa xong – là căn bản.
11. Các kiến trúc tốt nhất, yêu cầu tốt nhất, và thiết kế tốt nhất sẽ được làm ra bởi các nhóm tự tổ chức.
12. Đội phát triển sẽ thường xuyên suy nghĩ về việc làm sao để trở nên hiệu quả hơn, sau đó họ sẽ điều chỉnh và thay đổi các hành vi của mình cho phù hợp.



Đặc trưng của Agile

1. Tính lặp (iterative)
2. Tính tiệm tiến (Incremental) và tiến hóa (evolutionary)
3. Tính thích nghi (adaptive)
4. Nhóm tự tổ chức và liên chức năng
5. Quản lý tiến trình thực nghiệm (empirical Process Control)
6. Giao tiếp trực diện (face to face communication)
7. Phát triển dựa trên giá trị (value-base development)

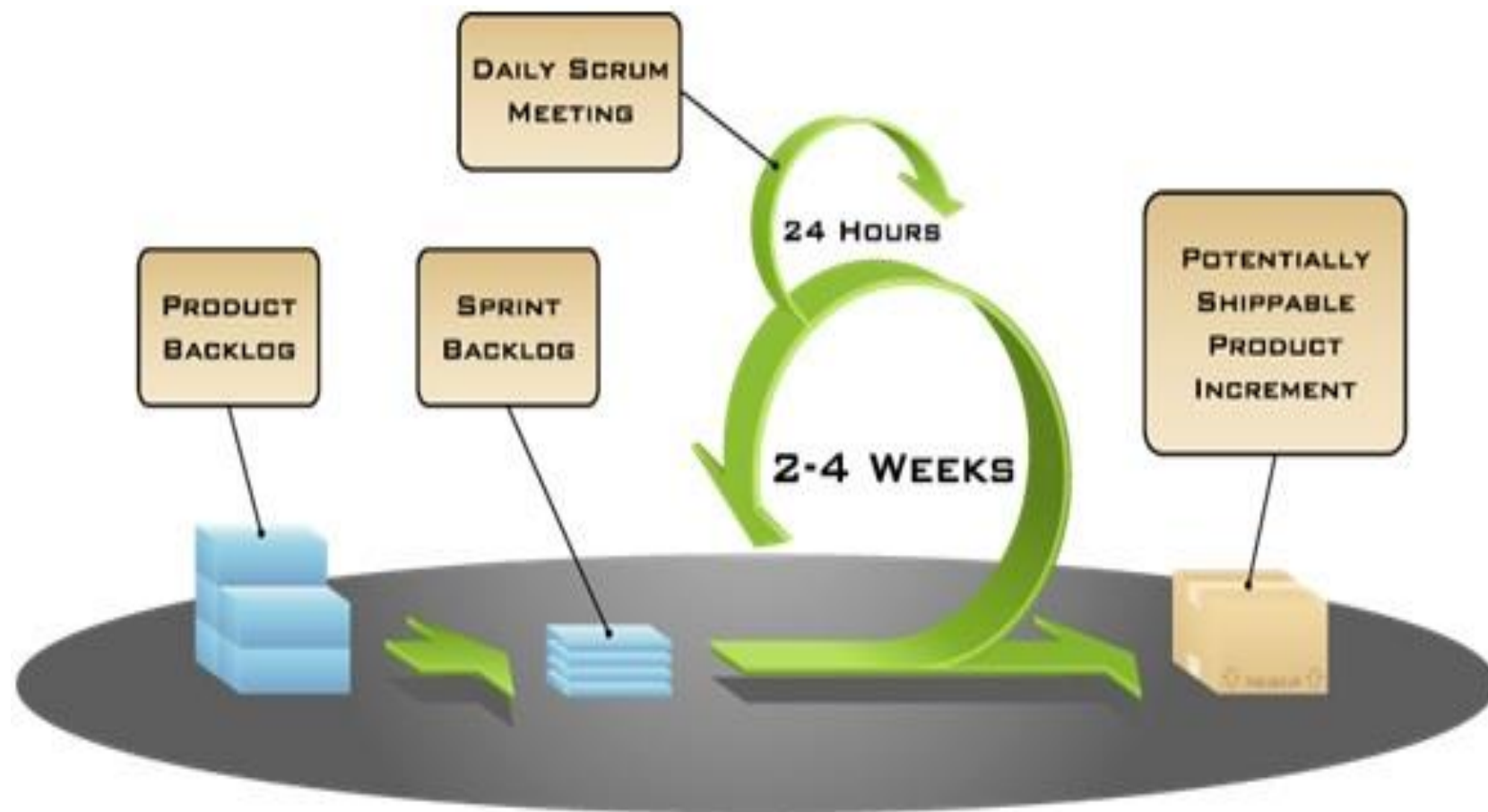


Quy trình Scrum

- Scrum chỉ là một trong nhiều phương pháp để thực hành Agile.
- Ngoài Scrum, chúng ta còn có nhiều phương pháp khác để trở nên “linh hoạt” như XP (Extreme Programming), Lean, Kanban, v.v Tuy nhiên, Scrum được ứng dụng phổ biến nhất hiện nay trong Agile.
- Là một phương pháp phát triển nhanh, trong đó tập trung đặc biệt vào làm thế nào để quản lý công việc trong một môi trường phát triển theo nhóm.



Quy trình SCRUM





Quy trình SCRUM

- Ví dụ về một cách để tạo sprint backlog

Ước tính Khối lượng công việc Còn lại vào cuối Ngày...									
Hạng mục Product Backlog	Công việc của Sprint	Người thực hiện	Ước tính khối lượng công việc ban đầu	1	2	3	4	5	6
Là khách hàng, tôi muốn thêm một cuốn sách vào trong giỏ hàng	Chỉnh sửa cơ sở dữ liệu		5						
	Tạo trang web (UI)		8						
	Tạo trang web (nghiệp vụ Javascript)		13						
	Viết kiểm thử chấp nhận tự động		13						
	Cập nhật trang trợ giúp khách hàng		3						
	...								
Cải thiện hiệu suất xử lý giao dịch	Hợp nhất mã DCP và hoàn thành kiểm thử ở mức tăng		5						
	Hoàn thành máy đặt hàng cho pRank		8						
	Thay đổi DCP và bộ đọc để sử dụng API http của pRank		13						



Quy trình Scrum

- Scrum là một cơ cấu tổ chức có tính gia tăng và lặp lại cho các dự án và việc phát triển các ứng dụng.
- Nó cấu trúc việc phát triển trong các chu kỳ công việc gọi là sprint.
- Thời gian của các sprint là cố định, những chu kỳ lặp sprint không vượt quá 1 tháng.
- Sprint kết thúc vào một ngày cụ thể đã xác định trước cho dù công việc có hoàn thành hay không và chúng không bao giờ được mở rộng thêm.



Quy trình scrum

- Tại đầu mỗi sprint một nhóm đa chức năng lựa chọn các yêu cầu của khách hàng hay là các item từ một danh sách ưu tiên. Đội phát triển dự án cam kết hoàn thành các mục trên vào cuối sprint.
- Trong suốt sprint các item này không đổi. Vào mỗi ngày nhóm sẽ họp trong một thời gian ngắn để kiểm tra sự tiến bộ của mình và điều chỉnh các bước tiếp theo.



Quy trình scrum

- Tại cuối sprint nhóm sẽ xem xét lại sprint với các bên liên quan như khách hàng v..v và nắm được phản hồi từ khách hàng cũng như các bên liên quan để có thể kết hợp chặt chẽ trong sprint tiếp theo.
- Scrum nhấn mạnh vào việc cuối mỗi sprint sản phẩm đã hoàn thành có nghĩa là code đã được tích hợp, được test đầy đủ và có khả năng chuyển giao cho khách hàng.



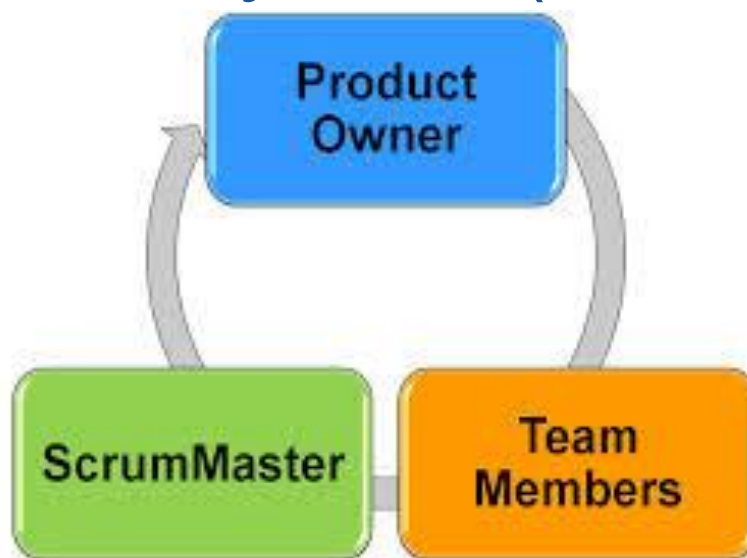
Quy trình Scrum

- Chủ đề chính của sprint là kiểm tra và thích nghi.
- Scrum nhấn mạnh vào từng bước ngắn của việc phát triển phần mềm.
- Scrum kiểm tra cả kết quả phần mềm và hiệu quả của thực tiễn hiện tại.
- Ba thành tố quan trọng tạo nên SCRUM:
 - Tổ chức nhóm dự án và vai trò
 - Quy trình và các sự kiện trong SCRUM
 - Các tài liệu đồ nghề trong SCRUM



Vai trò trong Scrum

- Trong Scrum có 3 vai trò: Chủ sản phẩm (Product Owner), Nhóm phát triển, và Scrum Master. Trong Scrum bạn sẽ không có vai trò Quản lý dự án (Project manager) hay Trưởng nhóm kỹ thuật (Technical leader).





Các sự kiện trong Scrum

- Sprint: là những chu kỳ nhỏ để phát triển sản phẩm. Mỗi Sprint sẽ có thời gian cố định được thống nhất, thường là 2 tuần và thường không hơn 4 tuần.
- Họp kế hoạch Sprint:
 - Diễn ra vào đầu mỗi Sprint bao gồm Chủ sản phẩm, Scrum Master và Nhóm phát triển.



Các sự kiện trong Scrum

- **Họp Scrum hằng ngày:**
 - Diễn ra sau họp kế hoạch Sprint
 - Buổi họp này thường diễn ra ngắn khoảng 15 phút và cố định về thời gian, địa điểm họp.
- **Họp sơ kết Sprint (Sprint Review Meeting):**
 - Vào cuối mỗi Sprint
 - Thành phần tham dự là tất cả những ai quan tâm đến sản phẩm.
 - Đánh giá sự thành công hay không của mục tiêu đặt ra.



Các sự kiện trong Scrum

- **Họp cải tiến Sprint (Sprint Retrospective Meeting):**
 - Thường diễn ra ngay sau buổi họp sơ kết Sprint và mất tầm khoảng 1-2 giờ thảo luận.
 - Nhóm sẽ đánh giá những việc mình đã làm và cách để làm cho nó tốt hơn.
 - Kết quả thực thi những cải tiến này sẽ được thảo luận trong buổi họp cải tiến của Sprint sau.



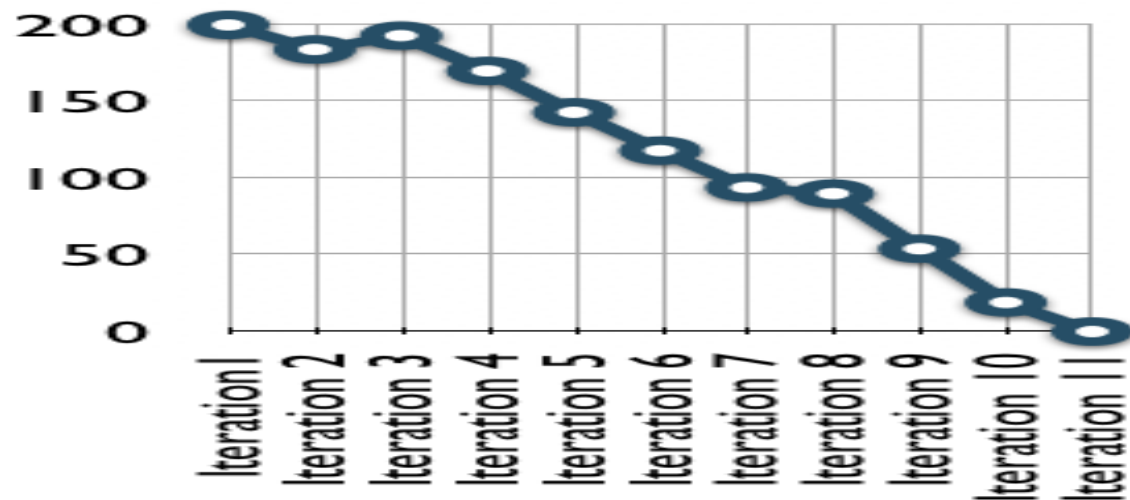
Đồ nghề trong Scrum

- Product Backlog:
 - Là một danh sách các đầu mục cần phải làm để phát triển sản phẩm bao gồm đủ loại như chức năng của sản phẩm, lỗi cần sửa, nghiên cứu công nghệ hay những việc công việc liên quan khác.
 - Những đầu mục này thường được mô tả dưới dạng “câu chuyện người dùng” (user story).



Đồ nghề trong Scrum

- Sprint Backlog: Là một danh sách các đầu mục mà nhóm cam kết hoàn thành trong Sprint.
- Biểu đồ Burndown (Burndown chart):
 - Được dùng để đo tiến độ của Sprint hay của dự án.





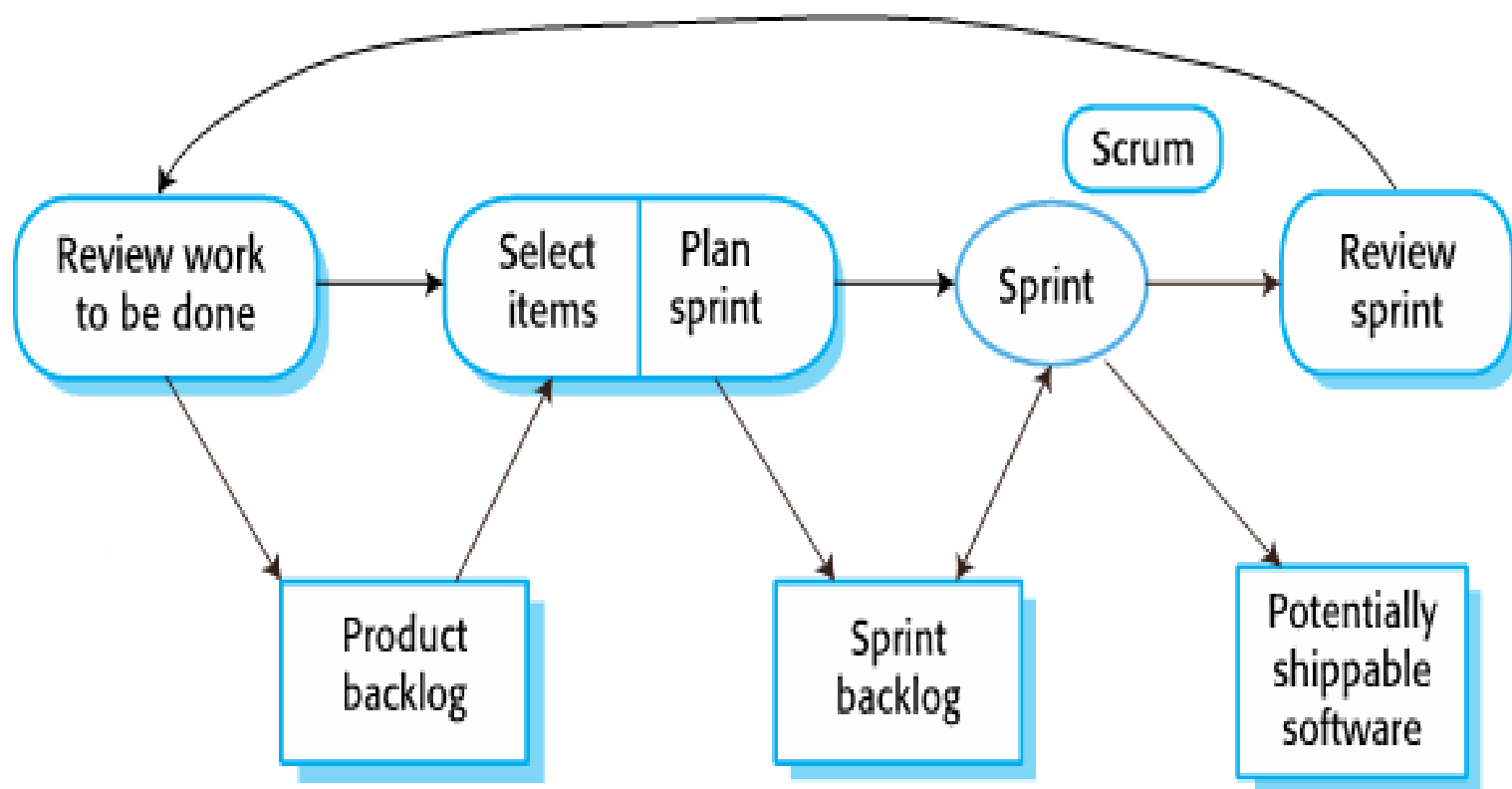
Đồ nghề trong Scrum

- Bảng công việc:
 - Được dùng để hiển thị những “câu chuyện người dùng” trong Sprint.

Story	To Do		In Process	To Verify	Done
As a user, I... 8 points	Code the... 9	Test the... 8	Code the... DC 4	Test the... SC 6	Code the... DC 8
	Code the... 2	Code the... 8	Test the... SC 8		Test the... SC 8
	Test the... 8	Test the... 4			Test the... SC 6
As a user, I... 5 points	Code the... 8	Test the... 8	Code the... DC 8		Test the... SC 8
	Code the... 4	Code the... 6			Test the... SC 6



Chu kỳ Sprint





So sánh với các mô hình khác

Đặc điểm	Waterfall	Spiral	Scrum
Xác định các giai đoạn phát triển	Bắt buộc	Bắt buộc	Chỉ có giai đoạn lập kế hoạch và kết thúc
Sản phẩm cuối cùng	Được xác định trong quá trình lập kế hoạch	Được xác định trong quá trình lập kế hoạch	Xác định trong quá trình xây dựng dự án
Chi phí sản phẩm	Được xác định trong quá trình lập kế hoạch	Thay đổi cục bộ	Xác định trong quá trình xây dựng dự án
Ngày hoàn thành sản phẩm	Được xác định trong quá trình lập kế hoạch	Thay đổi cục bộ	Xác định trong quá trình xây dựng dự án
Đáp ứng với môi trường sử dụng	Trong kế hoạch ban đầu	Trong kế hoạch ban đầu	Xuyên suốt từ kế hoạch đến xây dựng và kết thúc
Kinh nghiệm trao đổi	Đào tạo trước cho đến khi bắt tay làm dự án	Đào tạo trước cho đến khi bắt tay làm dự án	Thực hiện trong quá trình làm dự án
Khả năng thành công	Thấp	Trung bình thấp	Cao



Nội dung



3. Các giai đoạn trong tiến trình kỹ nghệ phần mềm

- Giai đoạn xác định
- Giai đoạn phát triển
- Giai đoạn bảo trì



Giai đoạn xác định

- Giai đoạn này xác định làm cái gì?
- Bao gồm ba bước:
 - Phân tích ở mức hệ thống
 - Lập kế hoạch dự án phần mềm
(sẽ được học trong môn quản lý dự án)
 - Phân tích yêu cầu
(sẽ được học trong chương 2)



Phân tích ở mức hệ thống

1. Các hoạt động cơ bản trong tiến trình phân tích hệ thống.
2. Đặc tả hệ thống



I. Các hoạt động cơ bản trong tiến trình phân tích hệ thống

- Bước 1: Xác định nhu cầu
 - Người phân tích giúp khách hàng xác định các mục tiêu của hệ thống.
 - Người phân tích phải phân biệt được giữa “nhu cầu” của khách hàng và “điều mong muốn” của khách hàng.
 - Khi các mục tiêu tổng thể đã được xác định, người phân tích chuyển sang việc đánh giá các thông tin phụ.



I. Các hoạt động cơ bản trong tiến trình phân tích hệ thống

- Bước 2: Nghiên cứu khả thi
 - Khả thi về kinh tế: đánh giá về chi phí phát triển cần phải cân xứng với lợi tức cuối cùng hay lợi ích mà hệ thống được xây dựng đem lại.
 - Khả thi về kỹ thuật: khảo cứu về chức năng, hiệu suất và ràng buộc có thể ảnh hưởng tới khả năng đạt tới một hệ thống chấp nhận được.



I. Các hoạt động cơ bản trong tiến trình phân tích hệ thống

- Bước 2: Nghiên cứu khả thi (tiếp)
 - Khả thi về hợp pháp: quy định của pháp luật về sự xâm phạm, vi phạm hay khó khăn nào có thể gây ra từ việc xây dựng hệ thống.
 - Khả thi về phương án: đánh giá tính khả thi của phương án đã xác định để tiếp cận tới việc xây dựng hệ thống.



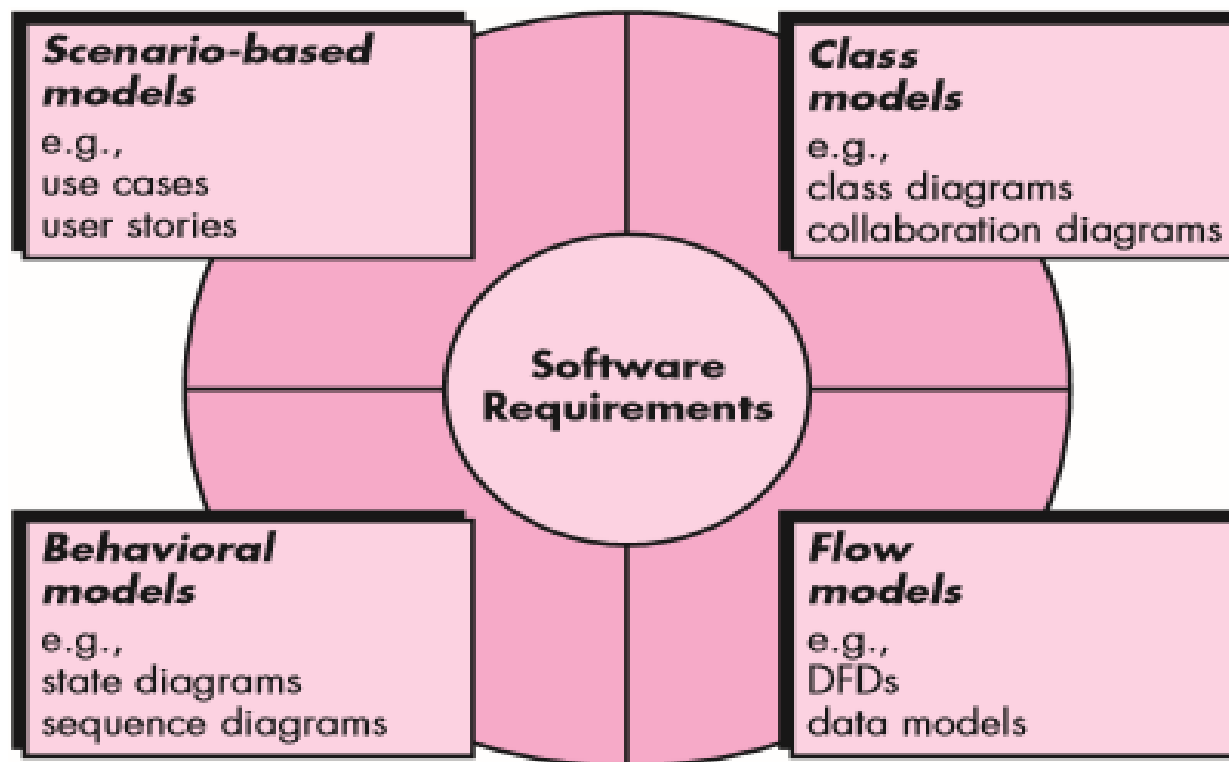
I. Các hoạt động cơ bản trong tiến trình phân tích hệ thống

- Bước 3: Mô hình hóa hệ thống
 - Việc mô hình hóa và mô phỏng hệ thống được sử dụng để loại bỏ những điều bất ngờ khi xây dựng hệ thống.
 - Những công cụ CASE được áp dụng trong các tiến trình kỹ nghệ hệ thống.



Mô hình hóa hệ thống

- Các yếu tố của mô hình phân tích: Phân tích cấu trúc và phân tích hướng đối tượng.





UML Diagram

Ví dụ:

- Use case diagram:
<https://www.smartdraw.com/use-case-diagram/>
- Activity Diagram:
<https://www.smartdraw.com/activity-diagram/>
- Class Diagram:
<https://www.smartdraw.com/class-diagram/>
- Sequence Diagram:
<https://www.smartdraw.com/sequence-diagram/>



II. Đặc tả hệ thống

- Bản đặc tả hệ thống:
 - Là các tài liệu làm nền tảng cho việc kỹ nghệ phần cứng, kỹ nghệ phần mềm, kỹ nghệ cơ sở dữ liệu và kỹ nghệ con người.
 - Mô tả về chức năng và hiệu suất của hệ thống và những ràng buộc hệ thống.
 - Quy định cả giới hạn cho từng phần tử hệ thống.
 - Mô tả thông tin (dữ liệu và điều khiển) vào/ ra khỏi hệ thống



II. Đặc tả hệ thống

- Xét duyệt đặc tả hệ thống: Tổ chức các cuộc họp cả hai bên.
 - Cuộc họp được tiến hành trong giai đoạn đưa ra các quan điểm quản lý áp dụng cho hệ thống, tiến hành đánh giá kỹ thuật về các phần tử và chức năng hệ thống
 - Cuộc họp phải đảm bảo: Tính đúng đắn của phạm vi dự án, chức năng, hiệu suất và giao diện. Phân tích rủi ro môi trường và tính khả thi. Người phát triển và khách hàng có cùng cảm nhận về mục tiêu hệ thống



Xét duyệt đặc tả hệ thống

- Về khía cạnh quản lý, tại giai đoạn đánh giá kỹ thuật, trong nhiệm vụ phân tích cần đều đặn trả lời các câu hỏi:
 - Nhu cầu kinh doanh của hãng đã được thiết lập chưa? Luận chứng hệ thống có nghĩa không?
 - Có cần môi trường (hay thị trường) riêng cho hệ thống đã được mô tả hay không?
 - Những phương án nào đã được xem xét?
 - Rủi ro phát triển cho từng phần tử hệ thống là gì?
 - Các tài nguyên đã có sẵn cho việc xây dựng hệ thống chưa?
 - Các giới hạn chi phí và lịch biểu có ý nghĩa gì không?

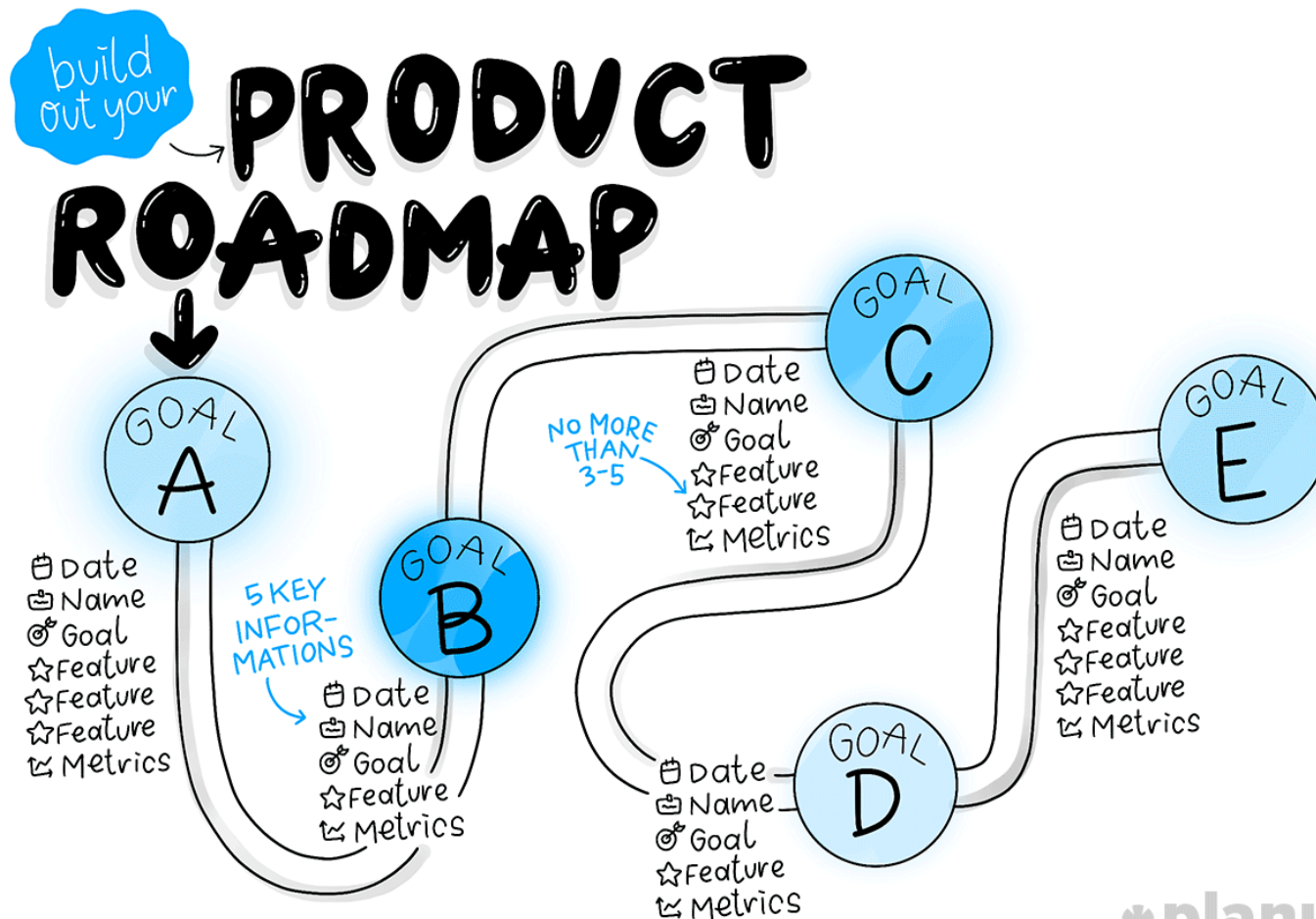


Xét duyệt đặc tả hệ thống

- Mức độ chi tiết trong giai đoạn đánh giá kỹ thuật phụ thuộc mức độ chi tiết trong nhiệm vụ xác định ban đầu nhưng phải bao gồm các vấn đề sau:
 - Về chức năng của hệ thống
 - Giao diện giữa các phần tử hệ thống và với môi trường
 - Các vấn đề độ tin cậy, hiệu suất và bảo trì
 - Cung cấp đủ nền tảng cho các bước kỹ nghệ phần cứng và phần mềm tiếp sau không?

Lập kế hoạch cho dự án phần mềm

- Xây dựng Product roadmap



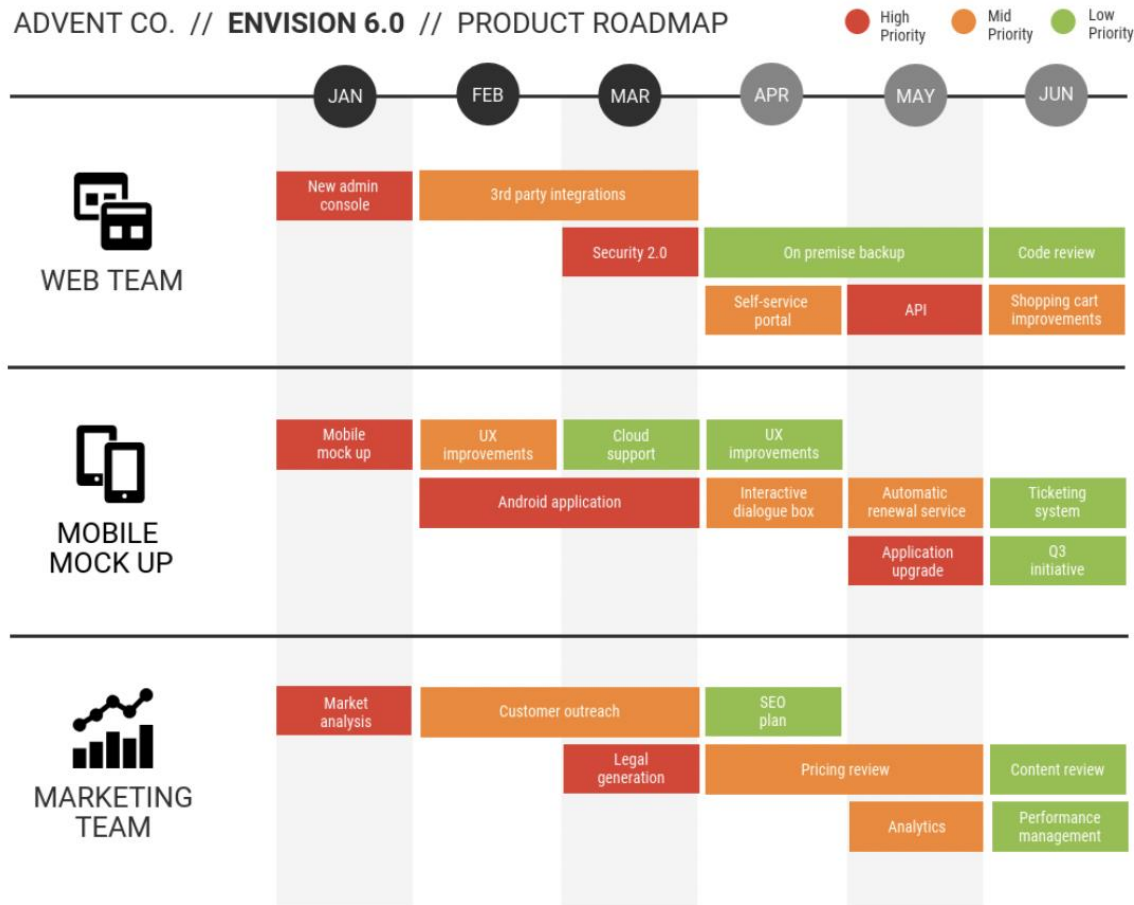
Lập kế hoạch cho dự án phần mềm

- Lập kế hoạch cho product roadmap



Lập kế hoạch cho dự án phần mềm

- Thiết kế product roadmap cho nhóm phát triển





Nội dung

3. Các giai đoạn trong tiến trình kỹ nghệ phần mềm

- Giai đoạn xác định
- Giai đoạn phát triển
- Giai đoạn bảo trì



Giai đoạn phát triển

- Giai đoạn này xác định làm như thế nào?
- Bao gồm ba bước:
 - Thiết kế
 - Thiết kế mức hệ thống
 - Thiết kế mức chi tiết phần mềm
 - Mã hóa
 - Kiểm thử phần mềm



Thiết kế hệ thống

Thiết kế hệ thống là gì?

- Là thiết kế cấu hình phần cứng và cấu trúc phần mềm (gồm cả chức năng và dữ liệu) để có được hệ thống thỏa mãn các yêu cầu đề ra ở giai đoạn phân tích.
- Có thể xem như Thiết kế cấu trúc (WHAT), chứ không phải là Thiết kế Logic (HOW)



Phương pháp thiết kế hệ thống

- Tổng quan về phương pháp thiết kế hệ thống:
 - Hai phương pháp hiện tại:
 - Hướng cấu trúc (hướng chức năng)
 - Hướng đối tượng
 - Phương pháp hướng khía cạnh



Quy trình thiết kế hệ thống

- Phân chia mô hình phân tích ra các hệ con
- Tìm ra sự tương tranh (concurrency) trong hệ thống
- Phân bổ các hệ con cho các bộ xử lý hoặc các nhiệm vụ (tasks)
- Phát triển thiết kế giao diện giữa chúng
- Chọn chiến lược cài đặt quản trị dữ liệu



Quy trình thiết kế hệ thống

- Để thiết kế hệ thống chúng ta cần:

→ – Mô tả cấu trúc tĩnh của hệ thống bằng một số biểu đồ:

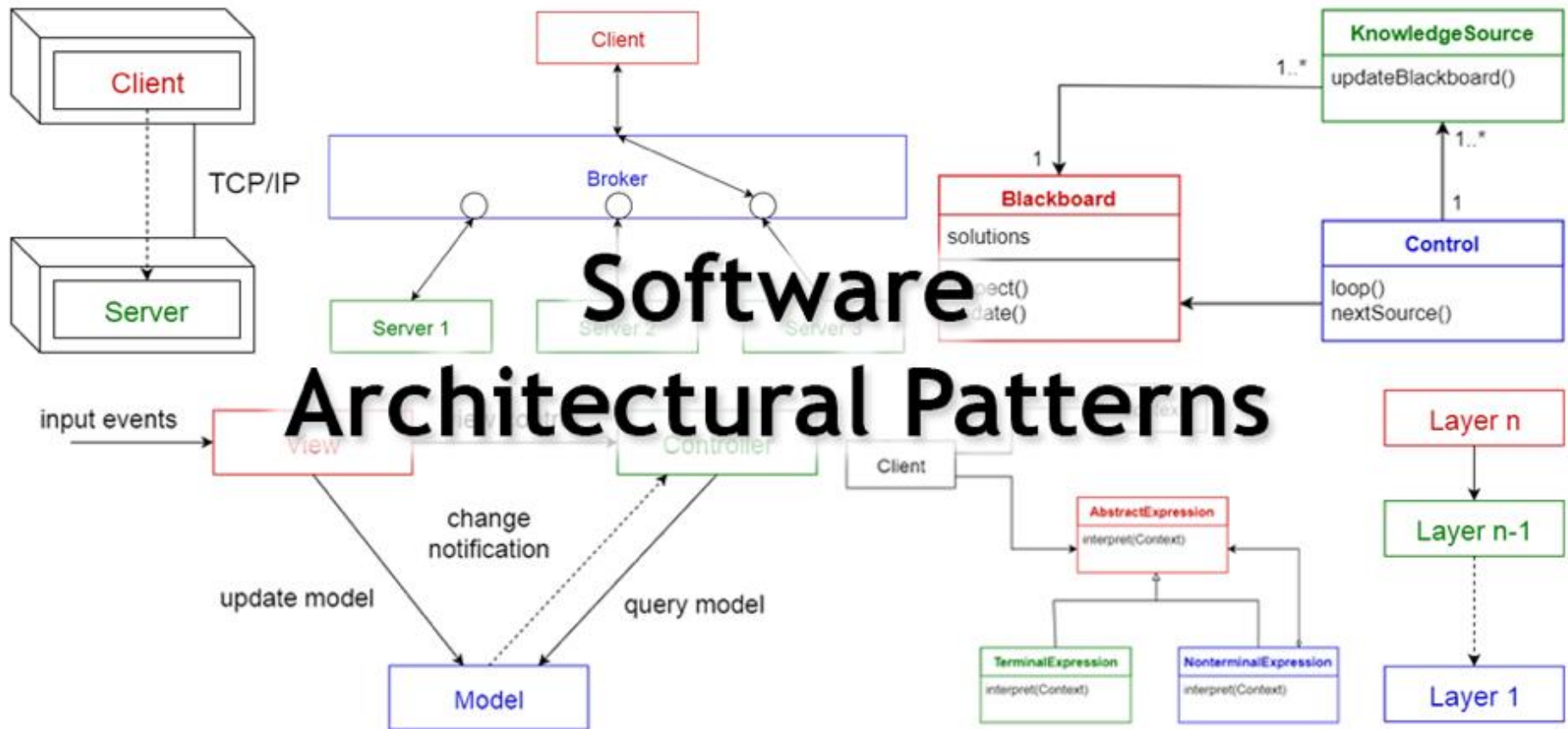
- Lựa chọn mô hình kiến trúc hệ thống. Từ đó
- vẽ biểu đồ thành phần, biểu đồ triển khai...

– Mô tả cấu trúc động của hệ thống bằng một số biểu đồ:

- Biểu đồ cộng tác Collaboration Diagram (UML 1.x)
còn gọi là Biểu đồ giao tiếp Communication Diagram
- Sơ đồ máy trạng thái State machine Diagram



Sử dụng các loại biểu đồ mô tả cấu trúc của hệ thống





Mô hình kiến trúc hệ thống

- Kiến trúc hệ thống là một hệ thống cấu trúc các thành phần cần thiết, đặc điểm, tính chất cũng như mối quan hệ giữa chúng.
- Thiết kế kiến trúc hệ thống :
 - Là giai đoạn đầu của quá trình thiết kế hệ thống
 - Là quá trình mô tả và phát triển hệ thống nhằm xác định:
 - Những hệ thống con hoặc mô-đun và
 - một khung làm việc giúp kiểm soát và giao tiếp giữa các hệ thống con/mô-đun này



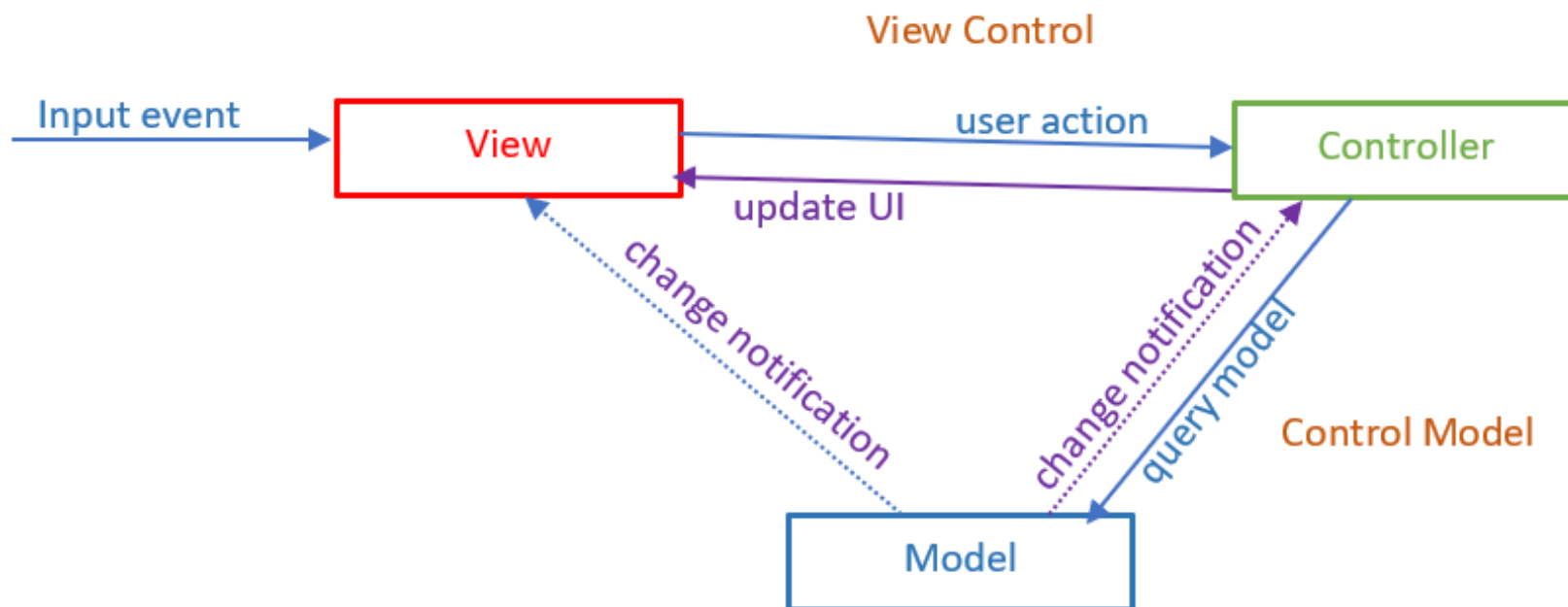
Mô hình kiến trúc hệ thống

- Một số mẫu mô hình kiến trúc hệ thống phổ biến:
 - Kiến trúc MVC (**Model-view-controller pattern**)
 - Kiến trúc phân tầng (**Layered pattern**)
 - Kiến trúc Khách – chủ (**Client-server pattern**)
 - Kiến trúc ống – lọc (**Pipe-filter pattern**)
 - Kiến trúc chủ - thợ (**Master-slave pattern**)
 - Kiến trúc dựa trên sự kiện (**Event-bus pattern**)
 - Kiến trúc kho (Repository Architecture)
 - Kiến trúc bảng đen (Blackboard Architecture)
 - ...



Mô hình kiến trúc phần mềm

- MVC





Biểu đồ thành phần

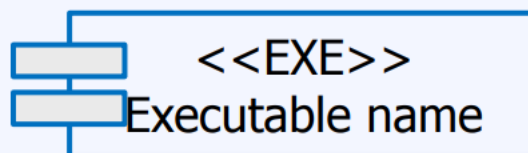
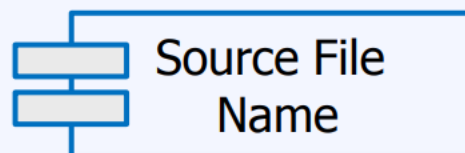
- Biểu đồ thành phần là bản vẽ cho biết cấu trúc của hệ thống theo thành phần phần mềm.
- Thành phần phần mềm được đóng gói độc lập, nó có thể được triển khai độc lập trên hệ thống và có khả năng tương tác với các thành phần khác khi thực hiện các chức năng của hệ thống





Biểu đồ thành phần

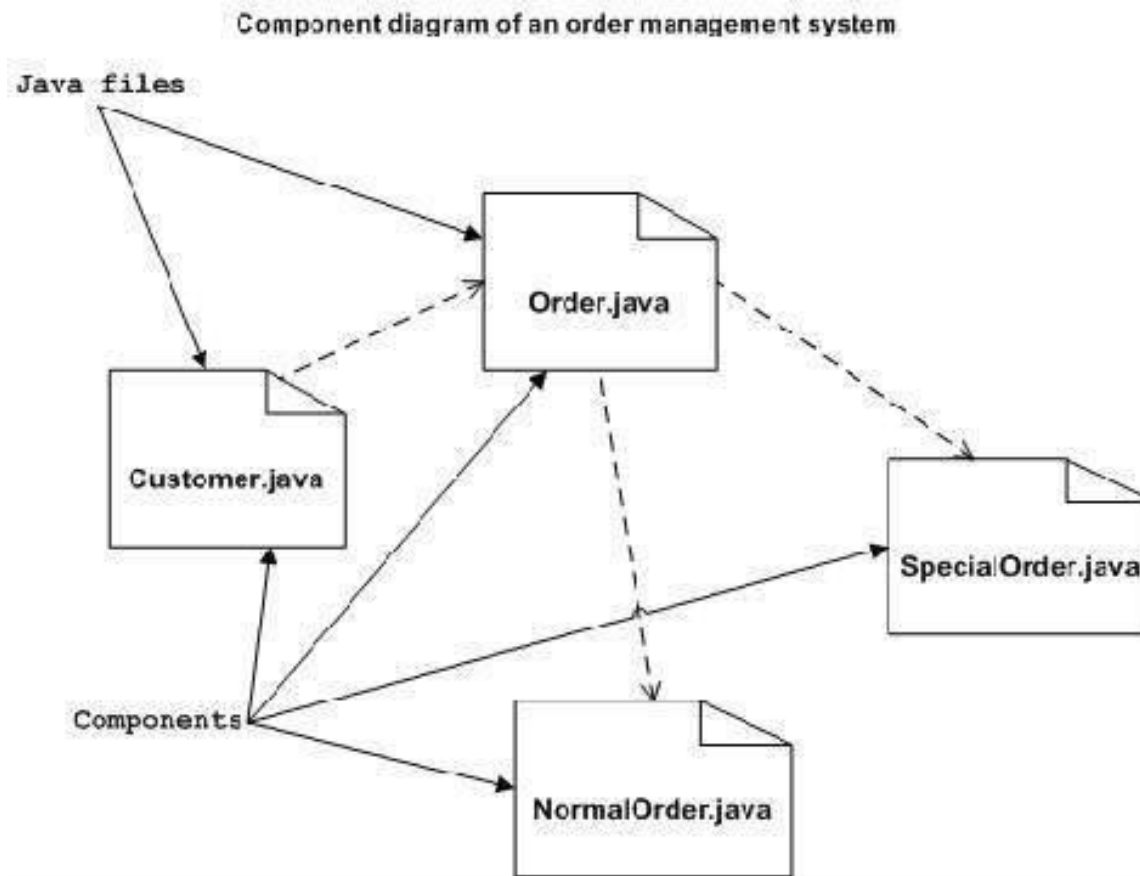
- Thành phần phần mềm giữ một chức năng rõ ràng trong hệ thống và có khả năng thay thế được.
- Một component có thể là:
 - Một source code component
 - Một runtime component
 - Một executable component





Biểu đồ thành phần

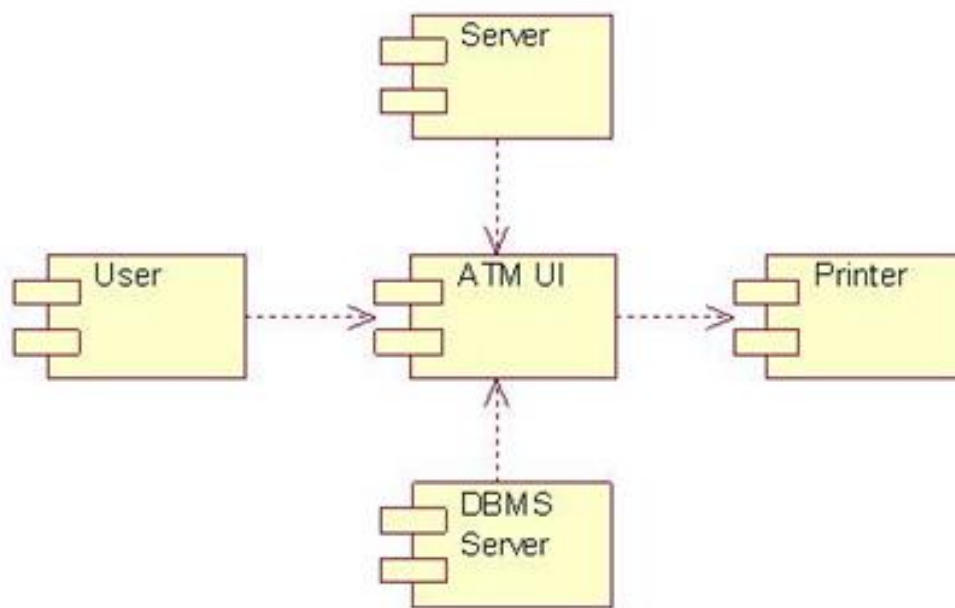
- Ví dụ 1:





Biểu đồ thành phần

- Ví dụ 2:



Component diagram cho ứng dụng máy ATM



Biểu đồ triển khai

- Biểu đồ triển khai giúp chúng ta xác định:
 - Sẽ triển khai hệ thống như thế nào?
 - Đặt các thành phần phần mềm lên hệ thống ra sao?
- Biểu diễn quan hệ vật lý giữa thành phần phần cứng và phần mềm như khi hệ thống đang thực hiện.
 - Biểu đồ triển khai có thể được sử dụng để chỉ ra thành phần nào có thể chạy trong node nào.



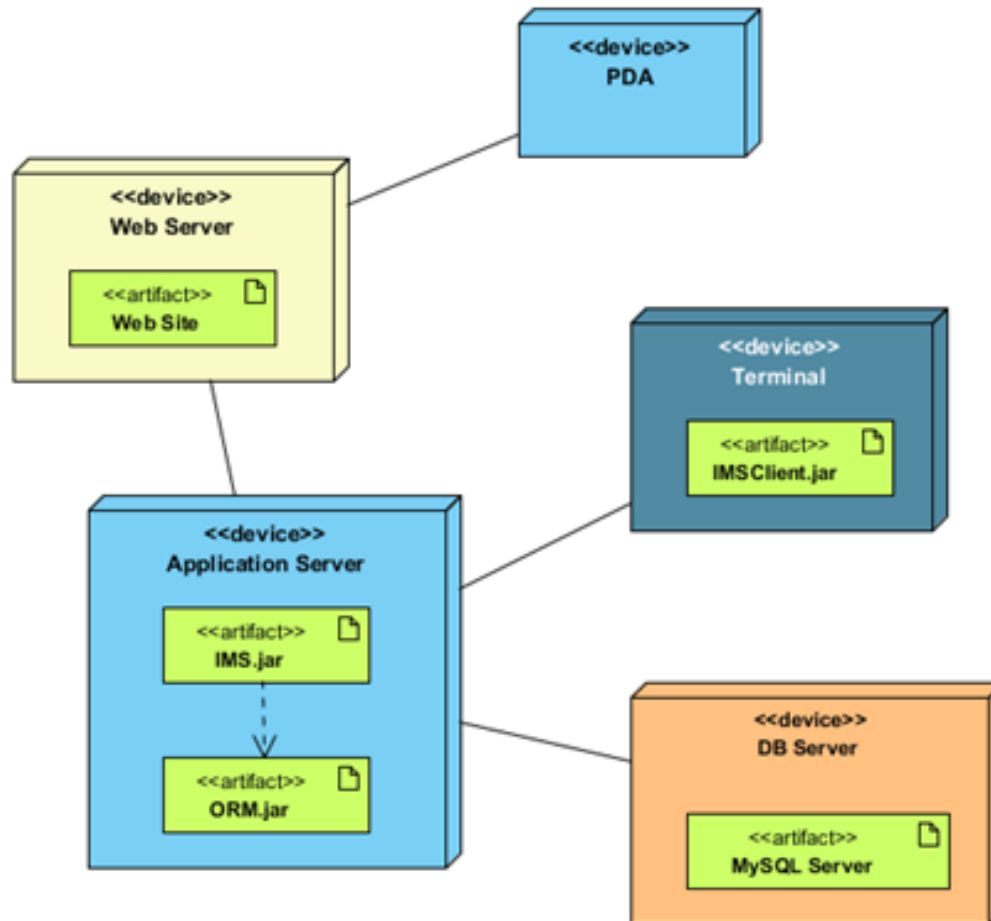
Biểu đồ triển khai

- Biểu đồ triển khai và biểu đồ thành phần có quan hệ gần nhau.
 - Biểu đồ thành phần để mô tả các thành phần, còn biểu đồ triển khai chỉ ra cách chúng được triển khai trên phần cứng.
- Mục đích của biểu đồ triển khai:
 - Hình dung ra cấu trúc liên kết của phần cứng
 - Mô tả thành phần phần cứng được sử dụng để triển khai các thành phần phần mềm.
 - Mô tả các node trong quá trình runtime.



Biểu đồ triển khai

- Ví dụ:





Biểu đồ triển khai

- Biểu đồ triển khai được sử dụng bởi các kỹ thuật viên hệ thống. Nó hiệu quả khi dùng để:
 - Thực thi hệ thống
 - Đánh giá thực thi hiệu năng của hệ thống, thực hiện khả năng chịu tải
 - Xem xét khả năng mở rộng của phần mềm
 - Dùng để bảo trì hệ thống
- Trước khi vẽ biểu đồ triển khai cần xác định
 - Các node (là một thành phần vật lý)
 - Các mối liên hệ giữa các node



Biểu đồ triển khai

- Trong đó các node là một thành phần vật lý, nó có thể là thiết bị phần cứng hoặc một môi trường nào đó mà các thành phần phần mềm được thực hiện.

Ví dụ như:

- Monitor
- Modem
- Caching Server
- Server



Quy trình thiết kế hệ thống

- Để thiết kế hệ thống chúng ta cần:
 - Mô tả cấu trúc tĩnh của hệ thống bằng một số biểu đồ:
 - Lựa chọn mô hình kiến trúc hệ thống. Từ đó
 - Vẽ biểu đồ thành phần, biểu đồ triển khai...
 - Mô tả cấu trúc động của hệ thống bằng một số biểu đồ:
 - Biểu đồ cộng tác/Biểu đồ giao tiếp (UML 2.x)
 - Sơ đồ máy trạng thái ...



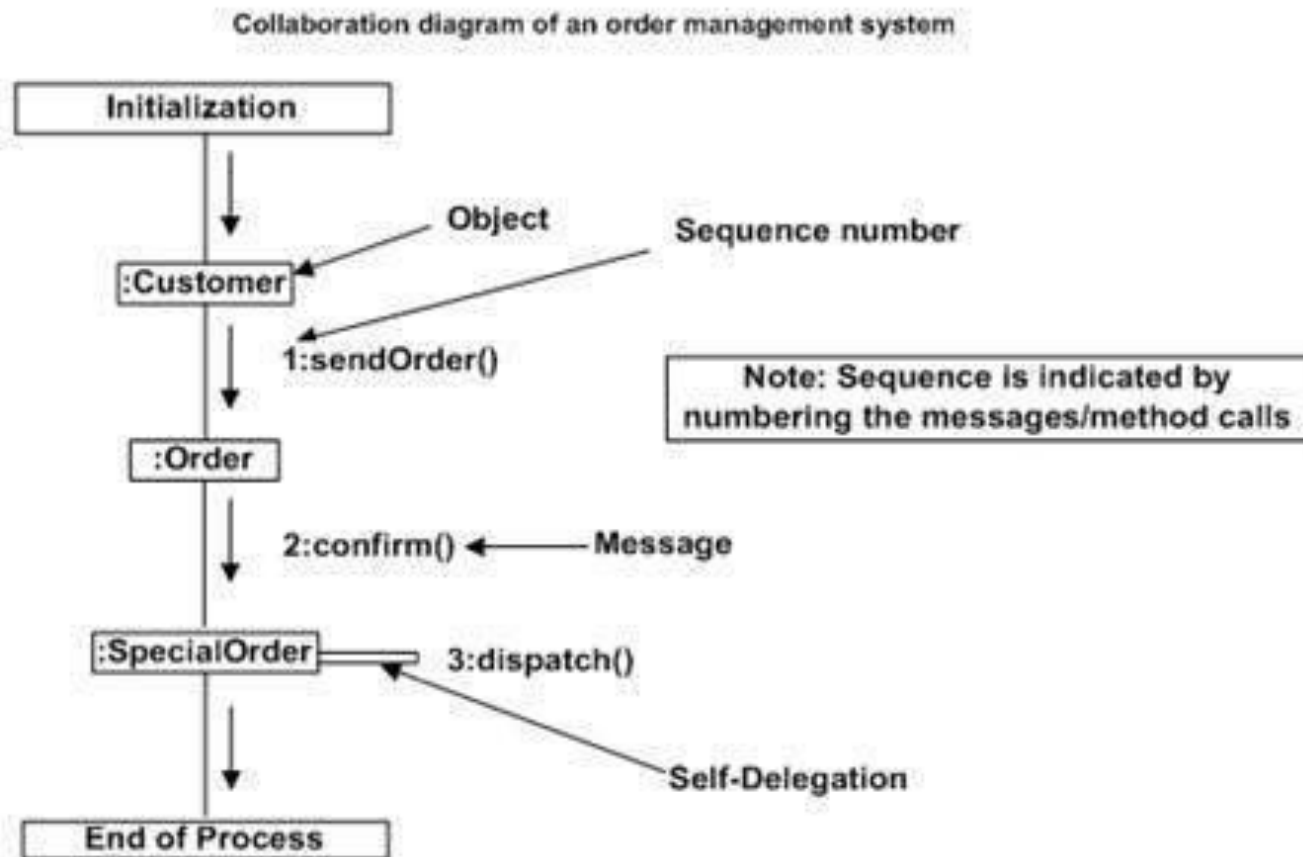
Biểu đồ cộng tác

- Tương tự như biểu đồ trình tự, biểu đồ cộng tác cũng chỉ ra luồng thực hiện nghiệp vụ cho từng chức năng của hệ thống.
- Biểu đồ cộng tác tập trung vào:
 - Quan hệ giữa các đối tượng
 - Cấu trúc tổ chức của các đối tượng
 - Hành vi của các đối tượng (điều mà sơ đồ trình tự thường tránh né)



Biểu đồ cộng tác

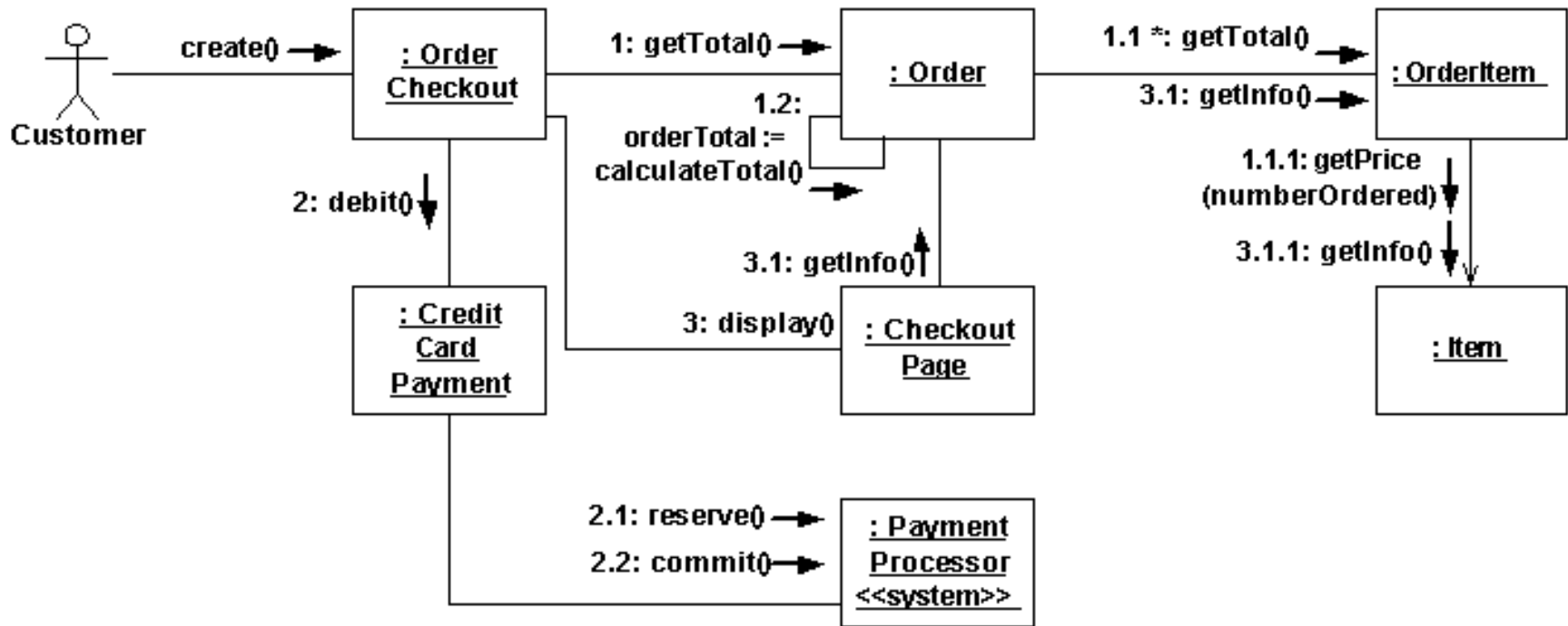
- Ví dụ: hệ thống quản lý chọn món





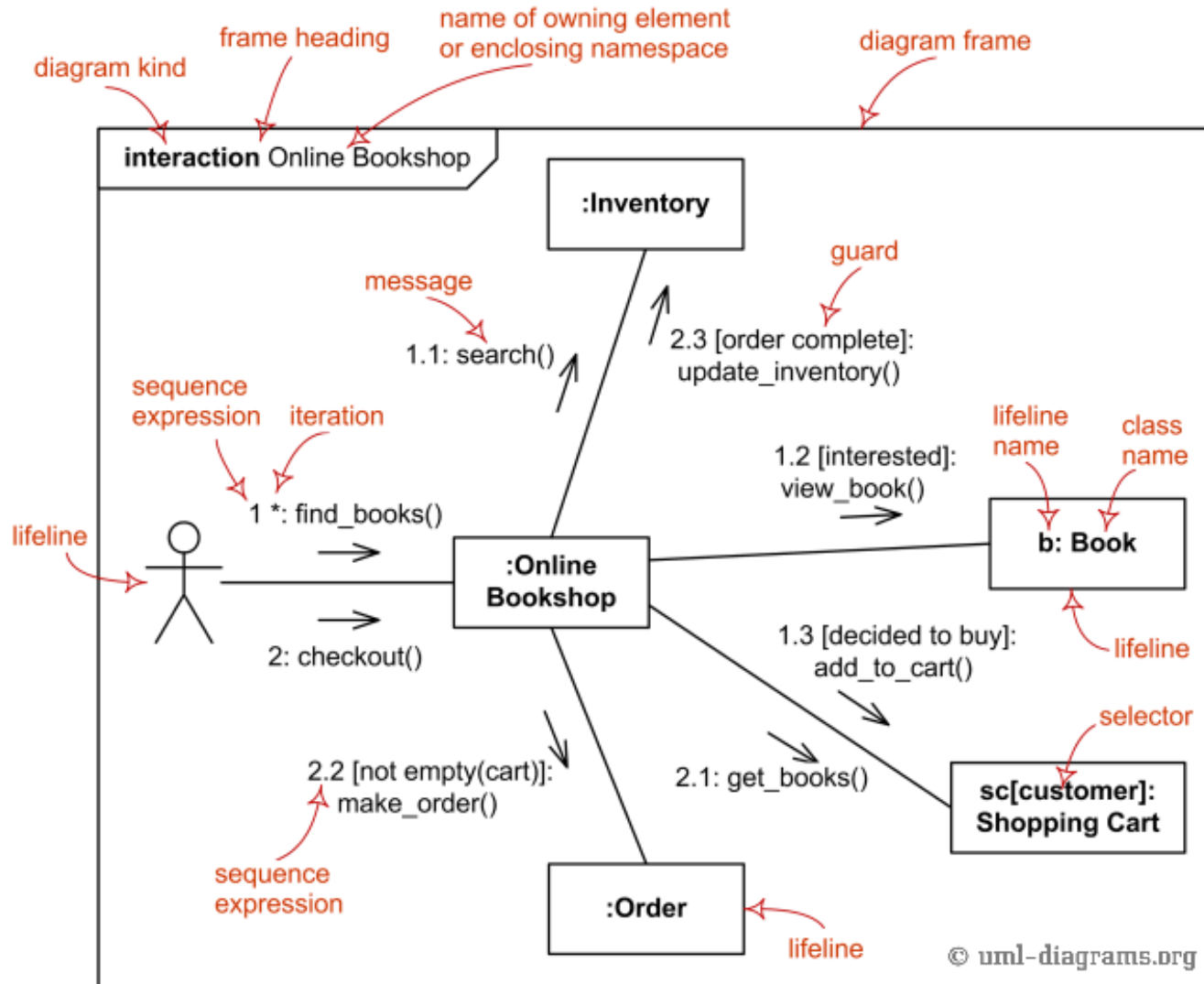
Biểu đồ cộng tác

- Ví dụ: Hệ thống quản lý đặt hàng





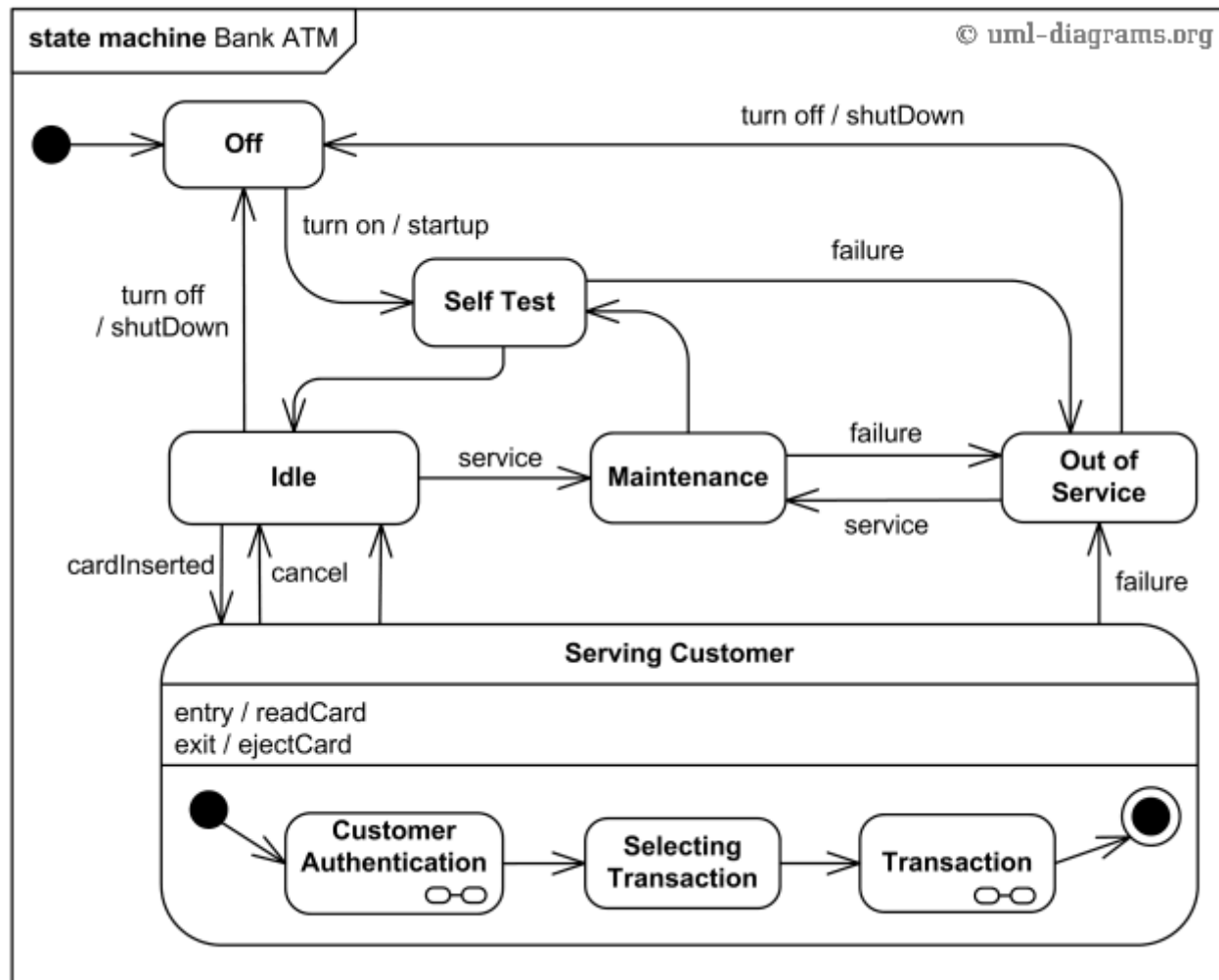
Biểu đồ giao tiếp/cộng tác





Mô tả cấu trúc động của hệ thống

- Sơ đồ máy trạng thái





Thiết kế mức chi tiết phần mềm

- Thiết kế chương trình:
 - Thiết kế chi tiết cấu trúc bên trong của phần mềm.
 - Thiết kế tính năng từng môđun và giao diện tương ứng giữa chúng.
- Vai trò của thiết kế:
 - Chuyển các mô hình ở mức phân tích thành các mô hình thiết kế đủ chi tiết để có thể lập trình.



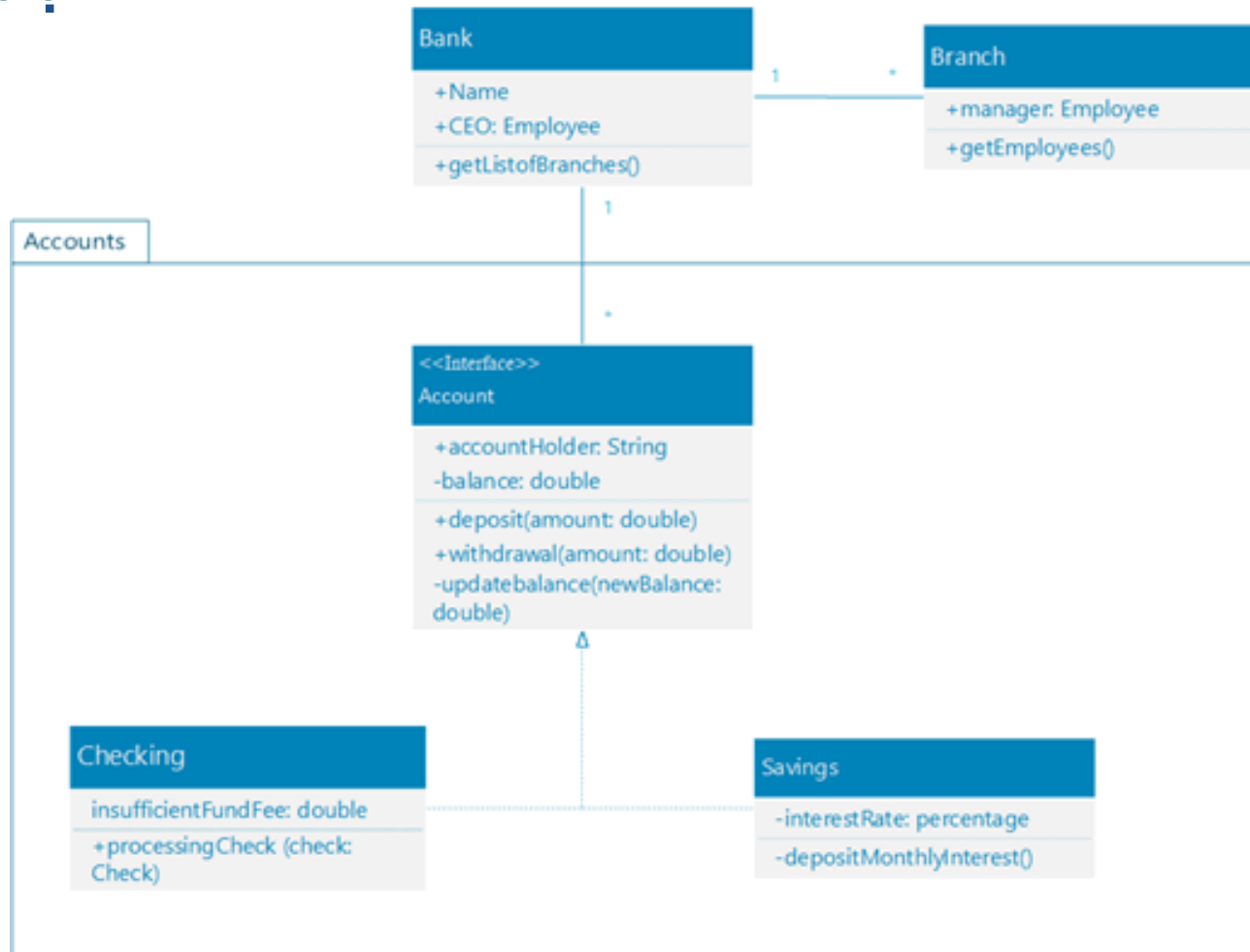
Một số loại biểu đồ

- Biểu đồ lớp:
 - Sử dụng biểu đồ lớp để tạo mô hình chung về cấu trúc của một ứng dụng.
 - Xác định:
 - các lớp của hệ thống,
 - các thuộc tính và phương thức của lớp
 - mối quan hệ giữa các đối tượng.
- (Chú ý: Nội dung bài học chỉ đề cập đến biểu đồ lớp)



Biểu đồ lớp

- Ví dụ:





Nhắc nhở bài tập lớn

- Quy trình PTTKHT hướng đối tượng
 - Mô hình hóa nghiệp vụ
 - Mô hình hóa hệ thống mức phân tích
 - Mô hình hóa hệ thống mức thiết kế

Chú ý: Sản phẩm đầu ra bao gồm một số biểu đồ được yêu cầu tìm hiểu qua các bài tiểu luận đã được giảng viên đưa ra.

- Tiểu luận 1+2+3



Nội dung

3. Các giai đoạn trong tiến trình kỹ nghệ phần mềm

- Giai đoạn xác định
- Giai đoạn phát triển
- Giai đoạn bảo trì



Giai đoạn bảo trì

- Nhiệm vụ của bảo trì là đảm bảo cho phần mềm vận hành tốt.
- Trong giai đoạn bảo trì có thể phải:
 - Sửa đổi
 - Bổ sung



5. Thảo luận

